

1. МИКРОПРОЦЕССОРНАЯ СИСТЕМА

Микропроцессорная система (МПС) представляет собой функционально законченное изделие, состоящее из одного или нескольких устройств, основу которого составляет микропроцессор.

Для того, чтобы микропроцессор (МП) выполнял свои функции ему необходимы дополнительные устройства:

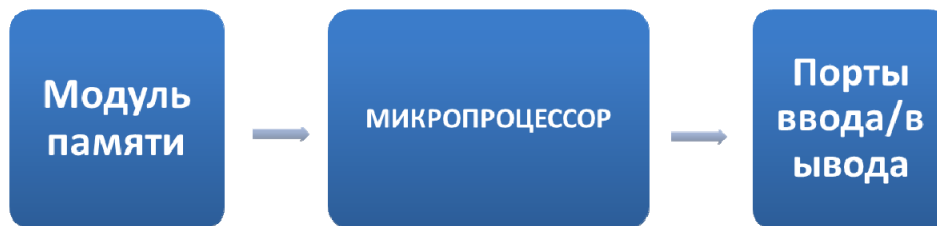


Рис.1 Структура микропроцессорной системы

Модуль памяти – это специализированное электронное устройство, предназначенное для записи, хранения и выдачи информации, представленной в виде цифровых кодов.

Порты ввода/вывода – это специализированные микросхемы, при помощи которых МПС может общаться с внешним миром.

Порт ввода – это специальное электронное устройство, на которое извне поступает электрические сигналы, предназначенные для управления микропроцессорным устройством.

Порт вывода выполняет обратную функцию, в него процессор записывает различные числа, которые затем поступают на внешнее устройство в виде электрических сигналов для управления этими устройствами

· Все 3 части МПС связаны между собой **шинами**-информационными каналами, которые объединяют все функциональные блоки МПС и обеспечивают обмен данными.

1.1 МИКРОПРОЦЕССОРЫ.

По **внутреннему устройству** в настоящее время наметилось два направления развития микропроцессоров:

- RISC-процессоры (процессоры с сокращенным набором команд);
- CISC-процессоры (процессоры с полным набором команд).

В процессорах с полным набором команд используется уровень микропрограммирования, обеспечивающий декодирование и выполнение команд микропроцессора. Команды микропрограмм называют микрокомандами. В этих процессорах формат команды не зависит от аппаратуры процессора. На одной и той же аппаратуре при смене микропрограммы могут быть реализованы различные микропроцессоры.

С другой стороны, смена аппаратуры никак не влияет на программное обеспечение микропроцессора. При разработке новых микросхем можно использовать аппаратные решения, никак не связанные с предыдущей микросхемой. Главное, чтобы микропрограмма эмулировала эту микросхему. То есть пользователь воспринимает новую микросхему как полный аналог старой. С его точки зрения у микропроцессора только увеличивается производительность, снижается потребление энергии, уменьшаются габариты устройств.

Неявным недостатком CISC-процессоров является то, что производители микросхем стараются увеличить количество команд, которые может выполнять микропроцессор, тем самым увеличивая сложность микропрограммы и замедляя выполнение каждой команды.

В RISC-процессорах декодирование и исполнение команды производятся аппаратно, поэтому количество команд ограничено минимальным набором. В этих процессорах понятия команда и

микрокоманда совпадают. Преимуществом этого типа процессоров является то, что команда может быть в принципе выполнена за один такт (не требуется выполнение микропрограммы), однако для выполнения тех же действий, которые выполняет одиночная команда CISC-процессора, требуется выполнение некоторой последовательности команд RISC-процессоров, иногда это последовательность довольно длинная. То есть выигрыш в быстродействии микропроцессора может быть сведен к нулю.

В большинстве случаев быстродействие у RISC -процессоров выше, чем у CISC -процессоров. Тем не менее, при выборе процессора нужно принимать в расчет все параметры в целом. Нужно учитывать, что тактовая частота RISC -процессора может оказаться значительно ниже, чем у CISC -процессора (особенно если в нем применяются специальные меры по повышению производительности), разрядность команды может оказаться выше, чем у CISC -процессора (что чаще всего и бывает). В результате общий объем исполняемой программы для RISC -процессора, как правило, превышает объем подобной программы для CISC -процессора.

Следующий признак классификации архитектур микропроцессоров — это система команд. По **системе команд** микропроцессоры отличаются огромным разнообразием, зависящим от фирмы-производителя. Тем не менее, можно определить две крайние архитектуры построения микропроцессоров:

- аккумуляторные микропроцессоры;
- микропроцессоры с регистрами общего назначения.

В микропроцессорах с регистрами общего назначения операнды математических операций могут находиться в любом внутреннем регистре. В зависимости от типа операции команда может быть одноадресной, двухадресной или трехадресной.

Принципиальным отличием аккумуляторных процессоров является то, что математические операции могут производиться только над одной особой ячейкой памяти, аккумулятором. Для того чтобы произвести операцию над произвольной ячейкой памяти, ее содержимое необходимо скопировать в аккумулятор, выполнить требуемую операцию, а затем скопировать полученный результат в произвольную ячейку памяти.

В настоящее время в чистом виде не существует ни та, ни другая архитектуры. Все выпускаемые в настоящее время процессоры обладают системой команд с признаками, как аккумуляторных процессоров, так и микропроцессоров с регистрами общего назначения.

Следующий признак, по которому классифицируются микропроцессоры, — это способ работы с системной памятью. По **способу работы с системной памятью** существует два основных принципа построения микропроцессоров:

- гарвардская архитектура;
- архитектура фон Неймана.

В гарвардской архитектуре принципиально различаются два вида памяти:

- память программ;
- память данных.

В гарвардской архитектуре принципиально невозможно производить операцию записи в память программ, что исключает возможность случайного разрушения управляющей программы в случае неправильных действий над данными. Кроме того, в ряде случаев для памяти программ и памяти данных выделяются отдельные шины обмена данными. Эти особенности определили области применения гарвардской архитектуры микропроцессоров. Она применяется в микроконтроллерах, где требуется обеспечить

высокую надежность работы аппаратуры. В сигнальных процессорах эта архитектура, кроме высокой надежности работы устройств, позволяет обеспечить высокую скорость выполнения программы за счет одновременного считывания управляющих команд и обрабатываемых данных.

Отличие архитектуры фон Неймана заключается в принципиальной возможности работы над управляющими программами точно так же, как и над данными. Это позволяет производить загрузку и выгрузку управляющих программ в произвольное место памяти процессора, которая в этой архитектуре не разделяется на память программ и память данных. Любой участок памяти может служить как памятью программ, так и памятью данных. Причем в разные моменты времени одна и та же область памяти может использоваться и как память программ, и как память данных. Для того чтобы программа могла работать в произвольной области памяти, ее необходимо модифицировать перед загрузкой, т. е. работать с ней как с обычными данными. Эта особенность архитектуры позволяет наиболее гибко управлять работой микропроцессорной системы, но создает принципиальную возможность искажения управляющей программы, что понижает надежность работы аппаратуры. Архитектура фон Неймана используется в универсальных компьютерах и в некоторых видах микроконтроллеров.

ОБОБЩЕННАЯ СТРУКТУРА МИКРОПРОЦЕССОРА

Рассмотрим обобщенную структуру микропроцессора показанную на рис.2.

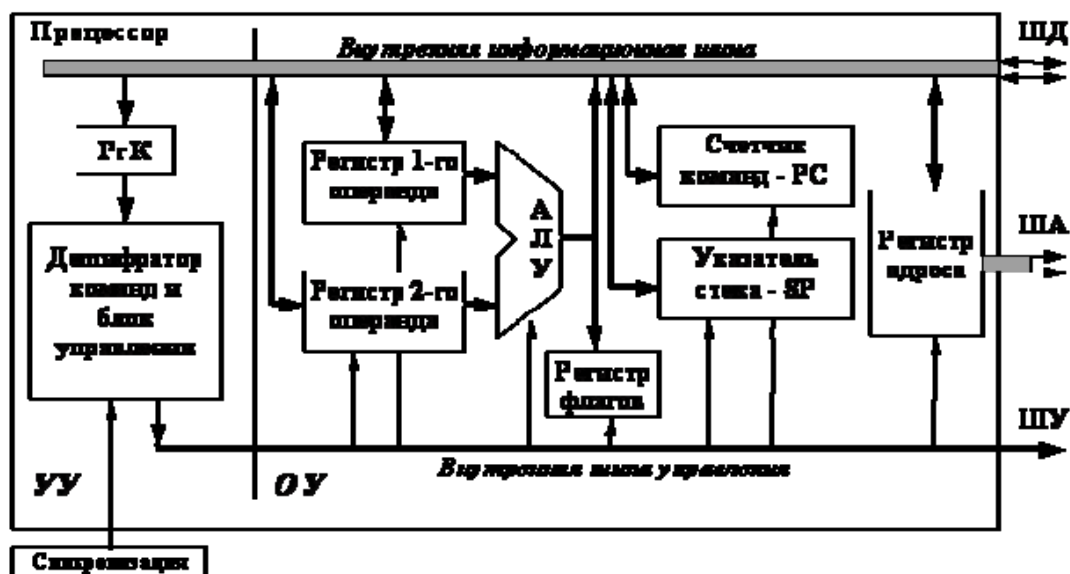


Рис. 2. Обобщенная структура микропроцессора

Процессор -это функциональный блок вычислительного устройства, предназначенный для реализации обработки цифровых данных и управления ходом этой обработки. Указанные действия выполняются процессором по командам, которые он автоматически считывает из памяти вычислителя.

В обобщенном виде функционирование процессора может быть представлено как циклическое чередование двух этапов – (1) выборки (чтения) команд из памяти и их дешифрации, и (2) выполнения команд.

Выборка (чтение) команд является автоматическим процессом, происходящим под воздействием импульсов от генератора тактовых импульсов (ГТИ), и не зависит от программиста в смысле механизма реализации, который жестко определяется аппаратной структурой процессора.

Дешифрация команды представляет собой процесс формирования последовательности управляющих сигналов для всех узлов процессора и других блоков вычислителя на основе информации (т.е. кода), содержащегося в команде.

Действия, выполняемые в соответствии с командой, могут представлять собой арифметическую или логическую обработку данных,

пересылку данных, формирование адреса следующей команды или изменение режимов работы процессора. В любом случае эти действия определяются программистом в рамках имеющейся в его распоряжении системы команд конкретного процессора. После выполнения действий, задаваемых командой, процессор автоматически переходит к выборке следующей команды из памяти.

Современные микропроцессоры существенно различаются набором функциональных блоков и связями между ними. Тем не менее, в структуре любого процессора можно выделить основные элементы, определяющие специфику процессора как управляющего центра вычислителя: устройство управления и операционное устройство. Устройство управления (УУ) предназначено для реализации выборки команд, их дешифрации, и на основе этого – для управления обменом и обработкой информации путем генерации последовательности управляющих сигналов. Операционное устройство (ОУ) служит для обработки цифровой информации (арифметические и логические операции, сдвиги, анализ чисел и т.п.).

Основным элементом для хранения информации внутри процессора являются регистры, которые выполняют функцию сверхоперативного ОЗУ с минимальным временем записи и считывания.

Регистр команд РгК (англ. IR – instruction register) используется для фиксации кода команды после считывания ее из памяти. Как правило, в этом регистре фиксируется код операции (КОП), т.е. часть кода команды, определяющая выполняемое действие и способ адресации операндов.

Регистры операндов служат для хранения данных в процессе их обработки, позволяют избегать постоянных обращений к памяти. В современных процессорах количество регистров операндов может достигать 10-15 штук. По сути, они образуют внутреннюю память процессора. В однокристальных микроконтроллерах количество регистров операндов доведено до нескольких десятков, и применительно к ним вводится понятие регистрового файла. Некоторые из регистров операндов

могут использоваться также для хранения или формирования адресов других операндов, т.е. на их основе реализуется механизм косвенной адресации данных в памяти. Данные, размещенные в регистрах операндов, поступают на обработку в арифметико-логическое устройство (АЛУ). В некоторых типах процессоров один из регистров операндов всегда является и приемником результата операции в АЛУ – такой регистр принято называть регистром-аккумулятором.

Счетчик команд (англ. PC - programmingcounter) - регистр, в котором при выборке или выполнении текущей команды формируется адрес следующей команды. Модификация содержимого регистра PC – это средство управления последовательностью выборки команд из памяти и, следовательно, управления ходом вычислительного процесса (т.е. реализация ветвлений в алгоритмах).

Указатель стека (англ. SP - stackpointer) - регистр, в котором при выполнении программы хранится адрес границы той области памяти, для которой программист использует принцип последовательного доступа к данным (так называемый протокол работы со стеком).

Регистр адреса - регистр, в котором формируется адрес любого устройства, внешнего по отношению к процессору (ячейки памяти или порта ввода-вывода), перед обращением к этому устройству. Данный регистр необходим, поскольку источником адресной информации могут являться различные регистры процессора. При этом регистр адреса играет роль накапливающего буфера, из которого адресная информация выдается на внешнюю шину адреса.

Регистр признаков или регистр флагов (англ. F - flags) - это элемент внутренней памяти, в котором отдельными битами фиксируются признаки, характеризующие результат операции, выполненной в АЛУ (нулевой результат, переполнение разрядной сетки и т.п.).

Арифметико-логическое устройство (АЛУ) - функциональный блок процессора, предназначенный для реализации действий по обработке данных.

Результат операции, выполненной в АЛУ, заносится в один из регистров или пересылается в память (в зависимости от команды). В регистре признаков автоматически формируются признаки, характеризующие этот результат.

Функционирование процессора всегда синхронизируется от внешнего генератора тактовых импульсов (ГТИ). Именно под влиянием импульсов от ГТИ устройство управления процессора автоматически реализует действия, связанные с выборкой команд из памяти и их дешифрацией.

Выполнение команды всегда занимает некоторое количество периодов тактовой частоты и состоит из последовательности элементарных действий процессора (выборка команды, чтение операнда, вычисление в АЛУ). Такие элементарные действия называются машинными циклами (МЦ). В течение каждого МЦ происходит генерация строго определенной комбинации управляющих сигналов для соответствующих узлов процессора и всей вычислительной системы.

1.2 ПАМЯТЬ

Память – это функциональная часть МПС, предназначенная для хранения информации. Запоминающий элемент – ячейка памяти, часть памяти, предназначенная для хранения наименьшей единицы информации. Основные параметры:

1. Ёмкость(объём) – определенное количество информации, которую можно хранить в памяти (бит, байт, слово). Бит – наименьшая единица информации, 1 двоичный разряд. Байт – 8 разрядов. Слово – 16 разрядов и более. $1K=2^{10}=1024$, $1M=2^{20}$, $1Г=2^{30}$, $1T=2^{40}$

2. ширина выборки – разрядность двоичных слов, хранящихся в памяти как единое целое (8-разрядное слово) - чаще всего для внутренней

1.1 ПАМЯТЬ

Память – это функциональная часть МПС, предназначенная для хранения информации. Запоминающий элемент – ячейка памяти, часть памяти, предназначенная для хранения наименьшей единицы информации.

Основные параметры:

1. Ёмкость(объём) – определенное количество информации, которую можно хранить в памяти (бит, байт, слово). Бит – наименьшая единица информации, 1 двоичный разряд. Байт – 8 разрядов. Слово – 16 разрядов и более. $1К=2^{10}=1024$, $1М=2^{20}$, $1Г=2^{30}$, $1Т=2^{40}$
2. ширина выборки – разрядность двоичных слов, хранящихся в памяти как единое целое (8-разрядное слово) - чаще всего для внутренней памяти, поэтому ширина выборки показывает организацию памяти и совпадает с разрядностью шины данных.
3. время доступа для чтения и\или записи – наименьшее время для чтения\записи в память и восстановления её свойств. Наиболее быстрая внутренняя память.
4. удельная плотность хранения информации – количество информации на единицу площади\объёма носителя. Наибольшая удельная плотность – у полупроводниковой памяти, наименьшая – магнитная лента.
5. удельная стоимость хранения информации – отношение ёмкости на стоимость носителя.

КЛАССИФИКАЦИЯ ПАМЯТИ

1. по назначению

- оперативная (ОЗУ). ОЗУ характеризуется тем, что время поиска ячейки, в которой хранится информация, не зависит от ее адреса (расположения в памяти), а разрядность обычно равна разрядности шины данных и кратна восьми. Память энергозависима.
- постоянная (ПЗУ). В ПЗУ хранятся системные программы, тесты, трансляторы, прикладные программы. Информация, записанная в ПЗУ, не

подлежит изменению и предназначена только для чтения. Отсюда иногда встречающееся название ПЗУ — ROM (от англ. read only memory — память только для чтения). Для размещения ПЗУ в адресном пространстве микропроцессора ему необходимо предоставить часть этого адресного пространства, свободную от ОЗУ. Это энергонезависимая память.

- Буферная память – используется для согласования по быстродействию различных элементов. Под буферной памятью типа FIFO ("первое записанное слово считывается первым" (First Input First Output - FIFO)) понимается запоминающее устройство, которое автоматически следит за порядком поступления данных и выдает их в том же порядке, допуская выполнение независимых и одновременных операций записи и считывания. На рис. 3 приведена структурная схема буферной памяти типа FIFO емкостью 64х4.

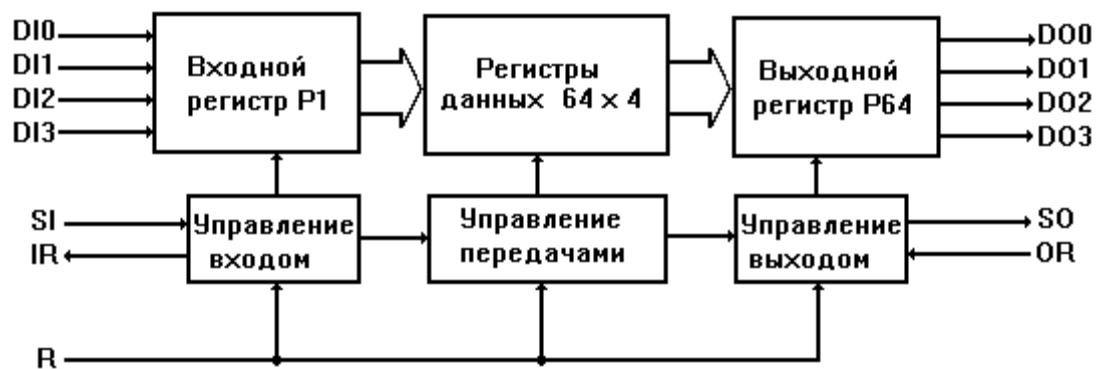


Рис. 3. Структурная схема буфера 64х4.

- Кэш-память – это промежуточный буфер с быстрым доступом, содержащий информацию, которая может быть запрошена с наибольшей вероятностью. Доступ к данным в кэше осуществляется быстрее, чем выборка исходных данных из более медленной памяти или удалённого источника, однако её объём существенно ограничен по сравнению с хранилищем исходных данных.

- Сверхоперативная память – это регистры общего назначения. Она имеет самое быстрое время доступа (для БИС ТТЛ 50—100 нс), ее объем от 8 до 32 ячеек и используется для временного хранения команд и данных, как правило, в течение выполнения одной или нескольких операций.

- Внешняя память – используется для долговременного хранения данных;

2. по физическому явлению:

- электронная (полупроводниковая) Выпускается в виде БИС.
- магнитная память,
- оптическая память(на основе оптических свойств материала);

3. по способу организации обмена память делится на память с произвольной выборкой и память с последовательной выборкой;

4. по способу организации поиска и выборки информации – на

- адресные запоминающие устройства(каждая информация имеет адрес) Адресная организация памяти предусматривает, что для обмена информацией необходимо указать адрес ячейки памяти и подать управляющий сигнал, указывающий направление передачи данных. Накопление осуществляется в виде матрицы ячеек, в каждой из которых хранится одно слово. Выборка нужной ячейки производится дешифратором на пересечении строк и столбцов. На вход дешифратора подается адрес, часть на строки, часть на столбцы. Данные поступают на усилитель записи – устройство, усиливающее и преобразующее сигнал с шины данных.

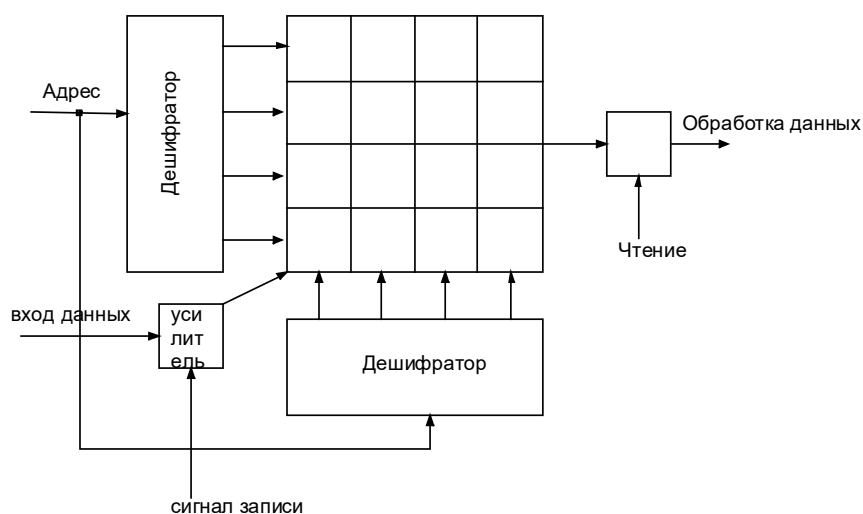


Рис.4 Адресная организация памяти.

- стековую память (запоминание по мере поступления) Стек – специально организованная память, состоящая из связанных ячеек , доступ

в которую осуществляется через одну из них. Виды стека - LIFO - Last in, first out; FIFO- -first in, first out.

Стековую память можно организовать в адресной, для этого в состав микропроцессора вводится специальный регистр – указатель стека, в него записывается адрес вершины стека(последней незаполняемой ячейки стека) при записи информации в вершину стека , указатель стека изменяется на единицу.

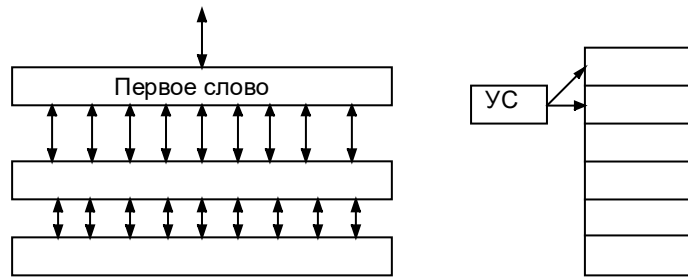


Рис.5 Стековая память

- ассоциативную память (поиск информации проводится по содержимым ячейки) является особым видом памяти, используемой в приложениях очень быстрого поиска. В ассоциативной памяти элементы выбираются не по адресу, а по содержимому.

5. По способу сохранения информации при выключении питания память делится на энергонезависимую и энергозависимую. При наличии двойного питания (дополнительного аккумулятора в корпусе микросхемы) ОЗУ будет энергонезависимой.

ОПЕРАТИВНАЯ ПАМЯТЬ

ОЗУ доступно для чтения и записи информации, и является энергозависимой (RAM) памятью.

ОЗУ делятся на статические и динамические.

Статическое ОЗУ (SRAM). В качестве запоминающего элемента используется. Триггер:

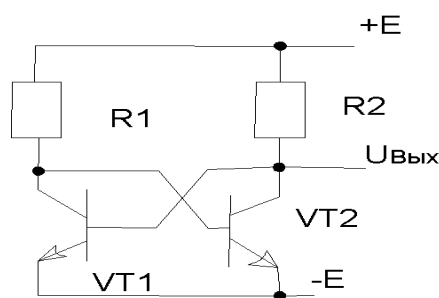


Рис. 6. Триггер. 1-е состояние (VT1 открыт, VT2 закрыт); 2-е состояние (VT1 закрыт, VT2 открыт), если VT2 закрыт – выход 1; если VT2 открыт - выход 0.

Многоэмиттерные транзисторы обладают свойством менять рабочий эмиттер изменением потенциала на нём. Рабочим является эмиттер, имеющий более низкий потенциал. Для записи 0 надо подать потенциал меньше 0 на 2-й эмиттер VT2

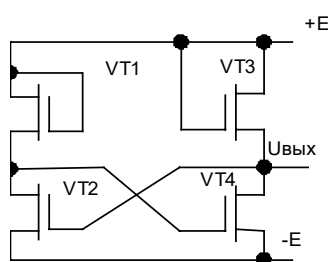


Рис.7 Использование многоэмитторного транзистора в качестве ячейки памяти.

Считывание и запись информации можно осуществить по одному из входов. Высокий потенциал на выходе при закрытом VT4, низкий потенциал на выходе при открытом VT4.

Запись информации осуществляется путём подачи на вход напряжения меньше $-E$, для включения ячейки в нулевое состояние.

Каждая микросхема содержит схему управления

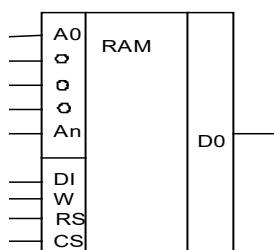


Рис.8 Обозначение ОЗУ. A0-An – адресные входы; DI – входы данных; W – сигнал записи, RS – сигнал чтения, CS – выбор кристалла.

Количество ячеек памяти определяется размерностью шины адреса.

Динамические ОЗУ (DRAM) . В отличие от статических, динамические ОЗУ основаны на свойстве конденсатора сохранять заряд. Основой динамических ОЗУ является униполярный транзистор к затвору которого подсоединен конденсатор. Если конденсатор зарядить, то на его концах остается разность потенциалов. В этом состоянии на выходе ноль.

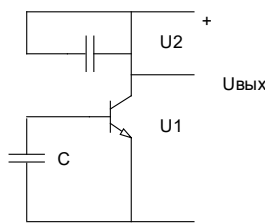


Рис.9. Ячейка памяти динамического ОЗУ.

В связи с тем, что ток затвора очень мал заряд сохраняется длительное время, что позволяет хранить информацию. Время хранения информации 10мс. Это мало – для сохранения информации. Необходимо постоянно ее восстанавливать. Процесс восстановления информации называется регенерация. Трег.= 60 мкс. Процент готовности микросхемы – 99.4%, 1 % - для восстановления. Требование регенерации является существенным недостатком динамических ОЗУ.

Достоинства:

Запоминающая ячейка реализуется на 1-ом транзисторе, что позволяет получить на одном кристалле большой объем памяти. Процесс регенерации осуществляется либо внешними схемами (по таймеру), либо внутри находится устройство для прерывания.

Обозначения динамических ОЗУ

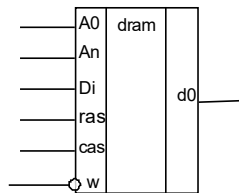


Рис.10 Обозначения динамических ОЗУ .A0...An- адресные сигналы
Di,D0- вход, выход,Ras – выбор строки, Cas-выбор столбца, W- запись

ПОСТОЯННАЯ ПАМЯТЬ

Типичное условное изображение микросхемы ПЗУ показано на рисунке 11.

Микросхемы ПЗУ любого типа имеют одинаковые группы сигналов.

- 1) входные сигналы шины адреса (ША),
- 2) один или несколько входов «выбор кристалла» CS (Chip Select),
- 3) выходные сигналы шины данных (ШД).

Если число разрядов адреса равно 15, то число ячеек памяти равно $2^{15} = 32768$. Разрядность ШД равна 8. Таким образом, ёмкость изображённой микросхемы равна 32768x8 или, как обычно говорят 32Kx8 – 32 килобайта.

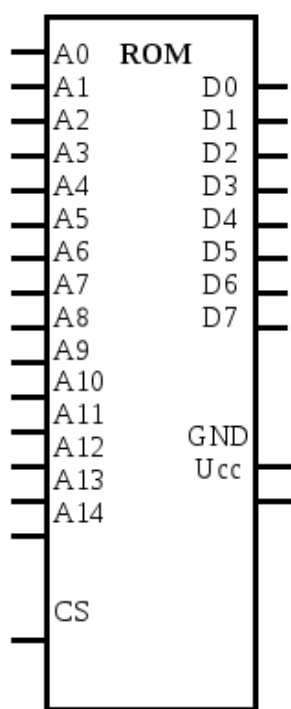


Рис.11. словное обозначение
микросхемы ПЗУ емкостью
32Кх8

При включении выходов (обычно $CS=«0»$) при подаче кода адреса на входы – на выходе ШД появится соответствующий код данных.

При $CS=«1»$ – выходы микросхемы в 3-м состоянии.

Соответствующим образом запрограммировав ПЗУ, можно реализовать любую логическую функцию (точнее – комбинационную схему). Входные переменные подавать на адресные входы, а с соответствующие значения функции.

Такова логика работы практически всех микросхем ПЗУ. Различие между ними заключается в ёмкости, то есть в количестве разрядов адреса и данных. Разница между различными типами ПЗУ заключается и в способе реализации элемента памяти. Ниже будут рассмотрены различные типы ПЗУ.

Масочные ПЗУ (МПЗУ)

Во многих видах ПЗУ элементом памяти является проводник – перемычка. Соединением её с лог. «0» или лог. «1», можно подавать на выход разряда ШД соответствующее значение сигнала. Перемычки могут быть изготовлены в кристалле полупроводника в виде резисторов, диодов или транзисторов.

Программирование ПЗУ заключается в формировании перемычек в соответствующих элементах памяти.

Программирование масочных ПЗУ (МПЗУ) производится на заводе-изготовителе путём изготовления трафарета («маски»). Для каждого типа микросхем изготавливается своя маска, отсюда и название данного типа ПЗУ.

Программирование МПЗУ производится однократно и перепрограммирование её невозможно. Микросхемы масочных ПЗУ обозначаются буквами РЕ, например – 155РЕ21.

Микросхемы МПЗУ в принципе не позволяют каким-либо образом изменить хранящуюся в них информацию. Это является и достоинством, так как делает хранение информации более надёжным, и недостатком – так как потребитель не может внести в память свою информацию.

Обычно МПЗУ используют в качестве источника стандартных функций, например: кодов русского, латинского алфавита, арифметических знаков и цифр, других символов, некоторых функций (например, тригонометрических) и даже простых операционных систем. То есть. Другими словами, - информации, которая не является уникальной и может быть использована многими пользователями.

Возможно изготовление маски по заказу потребителя, но в этом случае цена её будет существенно выше.

Существуют модификации некоторых типов микросхем МПЗУ. В этом случае, они называются, например:

K568PE1-0001, K568PE1-0002 и т. д.

Программируемые ПЗУ (ППЗУ)

Микросхемы программируемых ПЗУ по принципу построения и функционирования аналогичны масочным ПЗУ, но имеют существенное отличие в том, что допускают программирование на месте своего применения пользователем. Технические средства для программирования этой операции достаточно просты и могут быть построены самим пользователем. Это обстоятельство в сочетании с низкой стоимостью и доступностью микросхем ППЗУ обусловило их широкое распространение в радиолюбительской практике. Выпускаемые отечественной промышленностью микросхемы ППЗУ в большинстве своем изготовлены по ТТЛШ технологии, и среди них преобладающее положение занимает серия K556.

Функциональный состав серии включает микросхемы емкостью до 64 Кбит со словарной 4- и 8-разрядной организацией с временем выборки 45-85 нс, и уровнем потребляемой мощности 0.6 - 1 Вт.

Небольшая часть микросхем ППЗУ выполнена по другим технологиям ИИЛ (KS41), (n-МДП K565), ЭСЛ (K500 K1500), КМДП (K1623). Микросхемы серии K1623 отличаются самым низким уровнем энергопотребления, но по быстродействию они существенно уступают микросхемам K556 серии. Для микросхем ППЗУ всех серий, кроме K500, K1500, K565, характерны такие свойства, как единое напряжение питания +5 В, наличие входных и выходных ТТЛ уровней напряжения лог. нуля (0.4 В) и лог. единицы (2.4 В).

Пример микросхемы ППЗУ 556РТ16 ёмкостью 8Кх8 показан на рисунке. 12.

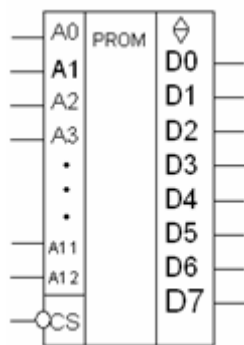


Рис. 12 Микросхема программируемого ПЗУ 556РТ16

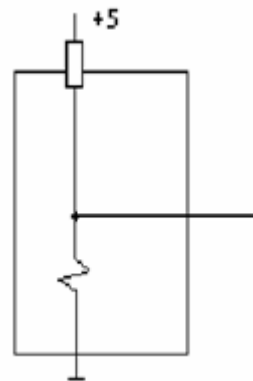


Рис.13.Элемент памяти ПЗУ

Уровень сигнала в ячейке памяти, как и в МПЗУ, задаётся перемычкой. Перемычки могут быть изготовлены из следующих материалов:

- титан – вольфрам
- силиций платины
- нихром
- поликристаллический кремний.

На рисунке 13 показан пример элемента памяти. В данном случае наличие перемычки означает лог. «0», т.к. она соединена с общим проводом, а отсутствие перемычки соответственно означает лог. «1».

Программирование ПЗУ

В исходном состоянии в микросхеме имеются все перемычки, т.е. записаны все нули. Операция программирования заключается в разрушении (пережигании) части плавких перемычек на поверхности кристалла импульсами тока амплитудой 30 - 50 мА в ячейках, в которых нужно изменить исходное состояние.

Пережигание происходит в соответствии с таблицей истинности, путём подачи более высокого напряжения $U = 10 - 12\text{В}$. В. Это в чём-то сходно с перегоранием предохранителей при возрастании тока выше допустимого значения.

Часто ППЗУ называют однократно программируемыми. Из принципа действия их понятно, что программировать их можно сколько угодно раз, но каждый раз пережигая новые перемычки, в то время, как восстановить их уже нельзя.

Некоторые микросхемы при поставке могут содержать несколько бит с уже изменённой информацией. Причиной этого является проверка микросхем на заводе на программируемость при изготовлении. Эти ячейки указываются в паспорте.

Пережигание перемычек производится по одной, чтобы не допустить перегрева и выхода из строя микросхемы.

Для программирования ПЗУ используются специальные устройства, называемые программаторами. В настоящее время существует много промышленных и радиолюбительских устройств для программирования ПЗУ.

После программирования нужно проверить информацию в ПЗУ, так как не все нужные перемычки могут быть разрушены. Для различных типов ППЗУ вероятность правильного программирования (коэффициент программирования) составляет: $K_{пр.} = 0,65 - 0,9$

При необходимости – производят повторное программирование. Возможно использование форсированного режима с использованием импульсов большей длительности.

Часто для повышения надёжности запрограммированное ППЗУ подвергают термотренировке – выдерживанию в термостате в течение определённого времени при высокой температуре (например – 168 часов при температуре +125 градусов). Если после термотренировки обнаружены ошибки – производят повторное программирование.

Нужно учитывать возможность восстановления перемычек, что на первый взгляд кажется невероятным. Причиной этого являются сложные явления миграции электронов на микроэлектронном уровне. Но эти явления не происходят в перемычках из силицида платины или поликристаллического кремния.

Программируемые логические матрицы

Разновидностью ППЗУ являются программируемые выжиганием плавких перемычек логические матрицы (ПЛМ).

Обычно они выполнены по ТТЛШ - технологии, например, К556РТ1 и К556РТ2 (рисунок 14), имеющие идентичные характеристики и конструктивные параметры, но отличающиеся типом выхода: у первой из микросхем выход с открытым коллектором, у второй на три состояния.

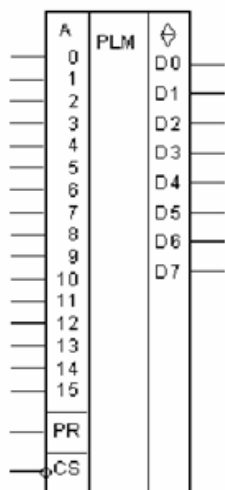


Рис.14. Микросхема
ПЛМ К556РТ2

Микросхемы ПЛМ также обозначаются буквами РТ и их условное изображение похоже на ИМС ППЗУ. Вместе с тем их устройство существенно отличается от ППЗУ.

Рассмотрим устройство ПЛМ на примере ИМС 556РТ2 (рисунок 14). ИМС 556РТ2 выполнена на основе технологии ТТЛШ. Уровни напряжений соответствуют уровням ТТЛШ. Упит = +5В.

Основу ПЛМ составляют матрицы И и ИЛИ. Матрица И выполняет операции конъюнкции над 16 входными переменными и их инверсными значениями, которые поступают на строчные шины матрицы. Требуемые логические произведения формируют на шинах столбцов путем выжигания ненужных перемычек между строками и столбцами (рисунок 15 - перемычки указаны «зигзагами»).

Число столбцов 48, следовательно, на выходе матрицы И можно получить до 48 логических произведений, в каждое из которых может входить до 16 переменных и их инверсий.

Матрица ИЛИ выполняет операцию дизъюнкции над логическими произведениями, сформированными матрицей И. Число выходов этой матрицы 8, поэтому она способна сформировать до восьми логических сумм, в каждую из которых может входить до 48 логических произведений.

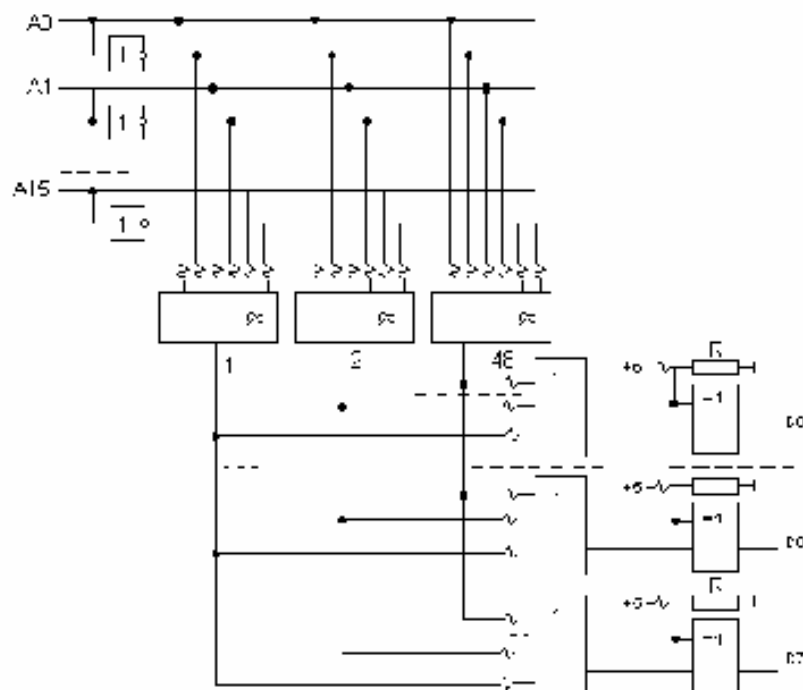


Рис.15.Электрическая схема ПЛИМ

Таким образом, возможности ПЛИМ характеризуются числом точек коммутации, равным в данном примере 1920.

Чтобы реализовать любую функцию, необходимо иметь $2^{16}=65536$ элементов И (число строк в таблице истинности). В 556РТ2 есть только 48, т.е. в таблице истинности может быть не более 48 единиц (или нулей –

учётом возможности инверсии на выходе). Т.е. ПЛМ не позволяет реализовать любую функцию.

Очевидно, что при одинаковом числе входных переменных (т.е. разрядов адреса), ПЛМ намного проще, чем ПЗУ.

Приведённая схема является сильно упрощённой, на ней не показаны некоторые узлы, обеспечивающие возможность программирования. Программировать ПЛМ намного сложнее, т.к. необходимо обеспечить несколько уровней программирования – для элементов И, ИЛИ и ИСКЛ. ИЛИ.

Программирование матрицы ИЛИ выполняется так же, как и матрицы И, путем выжигания «ненужных» перемычек. На выходах матрицы ИЛИ размещены программируемые усилители, которые в зависимости от состояния перемычки могут передавать значение выходной функции в прямой или в инверсной форме представления. Для программирования служат встроенные в микросхему узлы программирующей части, которые возбуждает разрешающий сигнал PR. Программирование осуществляют способом, аналогичным программированию ППЗУ, в три этапа: вначале программируют матрицу И, затем матрицу ИЛИ, затем выходные инверторы.

Итак, при одинаковом числе входных переменных, достоинством ПЛМ по сравнению с ППЗУ является более простая схема, а значит и меньшая цена. Недостатки – невозможность реализовать любую функцию и сложности при программировании.

Репрограммируемые ПЗУ

Репрограммируемые ПЗУ (РПЗУ) допускают многократное программирование пользователем. Число циклов программирования лежит в пределах 20 – 10000 и более. По англ. – EPROM (Erasable PROM., т.е. «стираемые ПЗУ»).

РПЗУ существует несколько типов:

- СППЗУ (EPROM) – стираемые программируемые ПЗУ;
- ЭСППЗУ (EPROM) – электрически стираемые программируемые ПЗУ;
- FLASH – память.

Электрически стираемые ПЗУ (ЭСППЗУ)

В ЭСППЗУ возможно стирание и программирование ПЗУ электрическими сигналами. В них используются элементы памяти на МОП-транзисторах, обладающие тем свойством, что пороговое напряжение отпирания транзистора может изменяться.

Элемент памяти со структурой nMHOП (показан на рисунке 16).

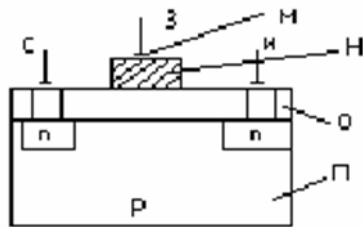
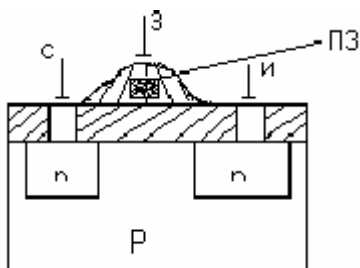


Рис.16 Элемент памяти nMHOП

Обозначения: С - сток, И – исток, З – затвор, М – металл, О – оксид кремния, П – полупроводник, Н - нитрид кремния.



Элемент памяти со структурой ЛИЗМОП. (МОП-транзистор с лавинной инжекцией заряда, рисунке 17) ПЗ – плавающий затвор

Рис.17. Элемент памяти ЛИЗМОП

Оба эти ЭП представляют собой разновидности МОП-транзисторов и работают сходным образом. В области Н (или на плавающем затворе – ПЗ) могут скапливаться электроны. Например, при подаче на затвор импульса достаточно большого положительного напряжения, электроны из области

подложки под действием электрического поля скапливаются в области затвора.

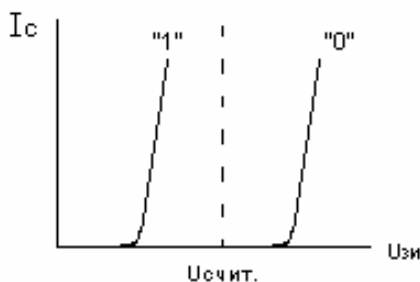


Рис.18. Характеристика транзисторов nMHOП и ЛИЗМОП

При этом происходит обеднение канала электронами, что приводит к увеличению порога отпирания транзистора, и его проходная характеристика смещается вправо.

При подаче отрицательного импульса происходит вытеснение электронов в область подложки, и характеристика смещается влево.

Для изменения порогового напряжения необходимо использовать импульсы большого напряжения (20-24 В). При обычном считывании характеристика транзистора не меняется. При чтении на затвор подаётся какое-то среднее напряжение считывания - $U_{\text{чит.}}$. В зависимости от места скопления электронов считывается значение «0» или «1» (рис 18)

Электрическая схема элемента памяти показана на рисунке 19.

ЭП – транзистор VT1 (nMHOП), а VT2 – транзисторный ключ, отпираемый при выборе соответствующего столбца матрицы элементов памяти.

Из сказанного выше следует, что для стирания и программирования информации необходимо применять напряжение 20-30 В различных полярностей. Однако используются физические принципы позволяющие применять напряжение программирования +20-24 В.

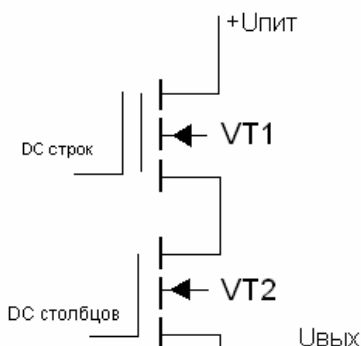


Рис. 19. Электрическая схема
элемента памяти пМНОП

Микросхемы рМНОП требуют 2-х источников питания: +5 и -20В, поэтому получили гораздо меньшее распространение.

Микросхемы ЭСППЗУ обозначается буквами РР. Рассмотрим одну из них на примере 558РРЗ. (рисунок 20).

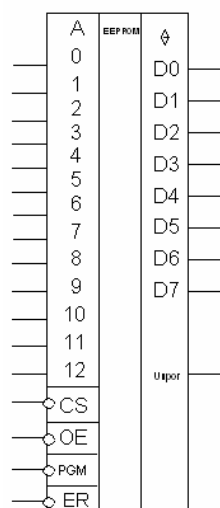


Рис. 20 ЭСППЗУ 558 РРЗ

Ёмкость микросхемы:

Микросхема 558РРЗ – 3У синхронного типа: запись во внутренний регистр адреса происходит по заданному фронту сигнала CS

OE =0 - разрешение чтения данных,

OE =1 - на выходе 3-е состояние.

При необходимости записи новых данных, сначала производят стирание всей информации, затем - запись.

Стирание происходит сразу для всей информации.

При этом во всех элементах памяти устанавливается «0».

Для стирания необходимо подать «0» на вход ER (Erase - стирание).

Запись производится в каждый элемент памяти., при этом в него записывается «1». Для записи необходимо подать на вход программирования (PR).логический «0». Запись установленного адреса ячейки памяти производится задним фронтом сигнала CS, а запись данных – задним фронтом сигнала OE.

Стираемые ППЗУ

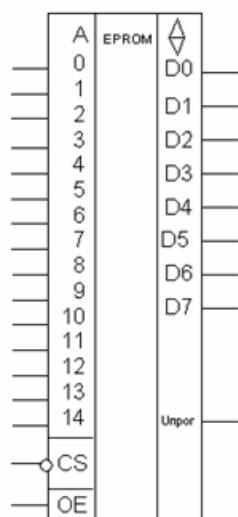


Рис. 21. СППЗУ 573 RP3

Стираемые ППЗУ (СППЗУ) во многом аналогичны ЭСППЗУ. В них используются элементы памяти ЛИЗМОП. Запись аналогична микросхемам ЭСППЗУ.

Основные отличия заключаются в том, что стирание производится ультрафиолетовым излучением (УФ). Для этой цели в микросхеме имеется прозрачное окно. Для стирания информации

необходимо поместить микросхему на расстояние нескольких сантиметров от УФ лампы. Для стирания информации необходимо примерно 30 минут. Затем окошко обычно заклеивают во избежание попадания света. После стирания в ППЗУ записаны все «1». Некоторые (редкие) СППЗУ стираются рентгеновским излучением.

В качестве примера рассмотрим микросхему 573РФ8 (микросхемы ППЗУ с УФ стиранием обозначаются буквами РФ).

Данная микросхема является асинхронным ЗУ (условное изображение показано на рисунке 21).

CS = «0» – разрешение записи.

OE = «0» – разрешение чтения.

В обычном режиме считывания – OE = CS = «0».

Достоинством ППЗУ с УФ стиранием является, прежде всего, более простая технология. Такие микросхемы ПЗУ являются самыми дешёвыми. В определённом смысле, достоинством является и то, что для стирания не требуется специальных схем и сигналов управления.

В остальном СППЗУ хуже ЭСППЗУ по следующим причинам:

1. Из-за попадания солнечного света или люминесцентного излучения возможно изменение информации. По этой причине СППЗУ с УФ

стиранием являются наименее надёжными. Часто их используют при проектировании в процессе отладки (как наиболее дешёвые), а в изделиях устанавливают ППЗУ или МПЗУ, имеющие аналогическую разводку контактов.

2. СППЗУ имеют значительно меньшее число циклов программирования (20 - 100) в отличие от ЭСПЗУ (свыше 10000).

3. СППЗУ не допускают избирательного стирания информации в отличие от некоторых ЭСППЗУ.

4. СППЗУ имеет значительно большее время стирания.

5. СППЗУ не допускают перепрограммирование без изъятия из контактного устройства.

FLASH-память

FLASH-память – наиболее современный тип полупроводниковых ЗУ, появился в 1989 г. Ежемесячный прирост её производства составляет 200%. Возможно, она вскоре вытеснит остальные виды ЗУ на микросхемах.

FLASH-память является разновидностью ЭСППЗУ, но использует особую технологию, позволяющую стирать информацию блоками, а не всю целиком, что делает процесс записи значительно более эффективным. FLASH-память использует ЭП с плавающим затвором. При стирании в микросхему (блок) заносится все «1».

Последние поколения FLASH-памяти позволяют производить запись, используя $U_{\text{прог}} = +5\text{В}$. При появлении микросхем FLASH-памяти, позволяющих стирать и записывать информацию в отдельных ячейках памяти, они практически будут работать как ОЗУ, но с возможностью хранения информации при отключении питания.

Многие микросхемы FLASH-памяти (а также некоторые DRAM) имеют встроенные автоматы (программируемые контроллеры), позволяющие упростить интерфейс работы с ними и использовать минимум управляющих сигналов.

1.3 АРИФМЕТИКО-ЛОГИЧЕСКОЕ УСТРОЙСТВО (АЛУ)

В общем случае многофункциональное АЛУ включает операционную часть (ОУ) и устройство управления (УУ), которое осуществляет вторичную дешифрацию кода команды и определяет выполняемую в АЛУ операцию (рис. 22)

Структура АЛУ:



Рис. 22 Структура АЛУ

Набор выполняемых в АЛУ операций должен обладать функциональной полнотой. Чтобы обеспечить функциональную полноту достаточно четырех операций:

- обращение к памяти для записи / чтения данных;
- инкремент / декремент;
- сравнение (реализует возможность условного перехода);
- останов работы устройства.

В первых процессорах количество операций ограничивалось шестнадцатью, теперь достигает нескольких сотен. Количество выполняемых операций является важной характеристикой АЛУ.

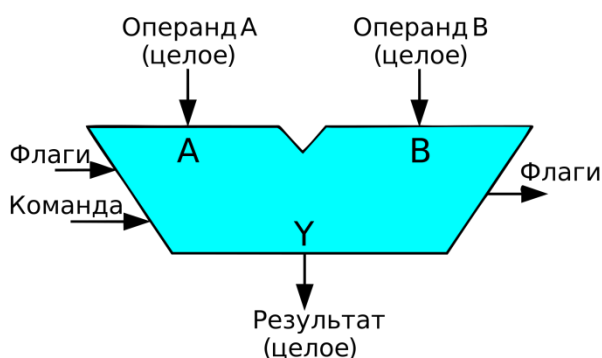


Рис. 23 Обобщённая блок-схема арифметико-логического устройства (АЛУ). Стрелками указаны входные и выходные слова. Флаги — признаки (например, результата сравнения операндов) выполнения предыдущей операции (вход) и результата выполнения текущей операции (выход). В одноместных операциях таких, например, как инверсия битов слова или битовый сдвиг второй операнд (В) не участвует в операции. Слово команды указывает необходимую операцию.

КЛАССИФИКАЦИЯ АЛУ

АЛУ можно классифицировать по ряду признаков, приведенных ниже.

Классификация по способу представления данных:

- с фиксированной запятой;
- с плавающей запятой.

Классификация по способу действия над операндами:

- последовательные АЛУ, где каждая операция выполняется последовательно над каждым разрядом;
- параллельные АЛУ, операция выполняется над всеми разрядами данных одновременно;
- последовательно - параллельные АЛУ, где слово данных делится на слоги, обработка данных ведется параллельно над разрядами слога и последовательно над слогами.

Классификация по использованию систем счисления:

- двоичная;

- двоично-десятичная;
- восьмеричная;
- шестнадцатеричная;
- и т.д.

Классификация по характеру использования элементов и узлов:

- блочные - для выполнения отдельных арифметических операций в структуру АЛУ вводят специальные блоки, что позволяет процесс обработки информации вести параллельно;
- конвейерные - в конвейерных АЛУ операция разбивается на последовательность микроопераций, выполняемых за одинаковые промежутки времени (такты) на разных ступенях конвейера, что позволяет выполнять операцию над потоком операндов каждый такт;
- многофункциональные - это универсальные АЛУ, выполняющие множество операций в одном устройстве. В таких АЛУ требуется настройка на выполнение данной операции при помощи кода операции.

Классификация по временным характеристикам.

По временным характеристикам АЛУ делятся на:

- синхронные - в синхронных АЛУ каждая операция выполняется за один такт.
- асинхронные - не тактируемые АЛУ, обеспечивающие высокое быстродействие, так как выполняются на комбинационных схемах.

Классификация по структуре устройства управления:

- АЛУ с жесткой логикой устройства управления;
- АЛУ с микропрограммным управлением.

Основные функции и характеристики АЛУ

Современные АЛУ выполняют:

- функции двоичной арифметики для данных в формате с фиксированной точкой;
- функции двоичной арифметики для данных в формате с плавающей точкой;

- функции арифметики двоично-десятичного представления данных;
- логические операций (в том числе сдвиги арифметические и логические);
- операции пересылки данных;
- работу с символьными данными;
- работу с графическими данными.

Основные характеристики АЛУ:

Основные характеристики АЛУ можно разделить на количественные и качественные.

Количественные характеристики:

Количественные характеристики определяют скорость выполнения операций, время выполнения одной операции, точность представления данных, количество выполняемых операций.

Среднюю скорость выполнения операций $V_{ср}$ в АЛУ можно определить как отношение $N(T)$ - количества операций, выполненных за отрезок времени T к данному отрезку времени:

$$ср = N(T)/T$$

Среднее время, которое АЛУ тратит на выполнение операции равно:

$$ср = 1/V_{ср}$$

Точность представления данных в АЛУ зависит от разрядной сетки АЛУ и выбранного формата данных.

Качественные характеристики АЛУ

К качественным характеристикам АЛУ относятся:

- структурные особенности АЛУ;
- форматы представления данных (с фиксированной или плавающей точкой);
- способы кодирования данных.

1.4 ШИНЫ МИКРОПРОЦЕССОРНОЙ СИСТЕМЫ

Обмен информацией в микропроцессорных системах происходит в циклах обмена информацией. Под циклом обмена информацией понимается временной интервал, в течение которого происходит выполнение одной элементарной операции обмена по шине. Например, пересылка кода данных из процессора в память или же пересылка кода данных из устройства ввода/вывода в процессор. В пределах одного цикла также может передаваться и несколько кодов данных, даже целый массив данных, но это встречается реже.

Циклы обмена информацией делятся на два основных типа:

- **Цикл записи (вывода)**, в котором процессор записывает (выводит) информацию ;
- **Цикл чтения (ввода)**, в котором процессор читает (вводит) информацию.

В некоторых микропроцессорных системах существует также цикл "чтение-модификация-запись" или же "ввод-пауза-вывод". В этих циклах процессор сначала читает информацию из памяти или устройства ввода/вывода, затем как-то преобразует ее и снова записывает по тому же адресу. Например, процессор может прочитать код из ячейки памяти, увеличить его на единицу и снова записать в эту же ячейку памяти. Наличие или отсутствие данного типа цикла связано с особенностями используемого процессора.

Особое место занимают циклы прямого доступа к памяти и циклы запроса и предоставления прерывания.

Во время каждого цикла устройства, участвующие в обмене информацией, передают друг другу информационные и управляющие сигналы в строго установленном порядке или, как еще говорят, в соответствии с принятым **протоколом обмена информацией**.

Длительность цикла обмена может быть постоянной или переменной, но она всегда включает в себя несколько периодов сигнала тактовой частоты

системы. То есть даже в идеальном случае частота чтения информации процессором и частота записи информации оказываются в несколько раз меньше тактовой частоты системы.

Чтение кодов команд из памяти системы также производится с помощью циклов чтения. Поэтому в случае одношинной архитектуры на системной магистрали чередуются циклы чтения команд и циклы пересылки (чтения и записи) данных, но протоколы обмена остаются неизменными независимо от того, что передается — данные или команды. В случае двухшинной архитектуры циклы чтения команд и записи или чтения данных разделяются по разным шинам и могут выполняться одновременно.

В системную магистраль (системную шину) микропроцессорной системы входит три основные информационные шины: адреса, данных и управления.

Шина данных — это основная шина, ради которой и создается вся система. Количество ее разрядов (линий связи) определяет скорость и эффективность информационного обмена, а также максимально возможное количество команд.

Шина данных всегда двунаправленная, так как предполагает передачу информации в обоих направлениях. Наиболее часто встречающийся тип выходного каскада для линий этой шины — выход с тремя состояниями.

Обычно шина данных имеет 8, 16, 32 или 64 разряда. За один цикл обмена по 64-разрядной шине может передаваться 8 байт информации, а по 8-разрядной — только один байт. Разрядность шины данных определяет и разрядность всей магистрали. Например, когда говорят о 32-разрядной системной магистрали, подразумевается, что она имеет 32-разрядную шину данных.

Шина адреса — вторая по важности шина, которая определяет максимально возможную сложность микропроцессорной системы, то есть допустимый объем памяти и, следовательно, максимально возможный размер программы и максимально возможный объем запоминаемых данных.

Количество адресов, обеспечиваемых шиной адреса, определяется как 2^N , где N — количество разрядов. Например, 16-разрядная шина адреса обеспечивает 65 536 адресов. Разрядность шины адреса обычно кратна 4 и может достигать 32 и даже 64. Шина адреса может быть односторонней (когда магистралью всегда управляет только процессор) или двусторонней (когда процессор может временно передавать управление магистралью другому устройству, например контроллеру ПДП). Наиболее часто используются типы выходных каскадов с тремя состояниями или обычные ТТЛ (с двумя состояниями).

Как в шине данных, так и в шине адреса может использоваться положительная логика или отрицательная логика. При положительной логике высокий уровень напряжения соответствует логической единице на соответствующей линии связи, низкий — логическому нулю. При отрицательной логике — наоборот. В большинстве случаев уровни сигналов на шинах — ТТЛ.

Для снижения общего количества линий связи магистрали часто применяется мультиплексирование шин адреса и данных. То есть одни и те же линии связи используются в разные моменты времени для передачи как адреса, так и данных (в начале цикла — адрес, в конце цикла — данные). Для фиксации этих моментов (стробирования) служат специальные сигналы на шине управления. Понятно, что мультиплексированная шина адреса / данных обеспечивает меньшую скорость обмена, требует более длительного цикла обмена (рис. 24). По типу шины адреса и шины данных все магистрали также делятся на мультиплексированные и немultipлексированные.

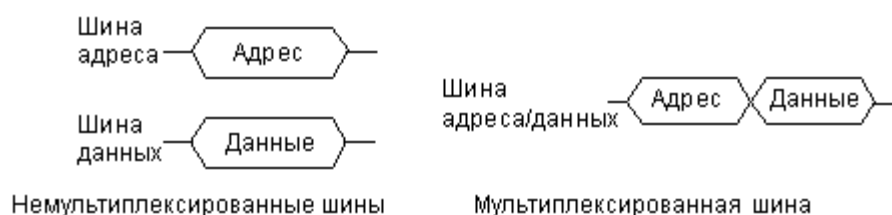


Рис. 24. Мультиплексирование шин адреса и данных.

В некоторых мультиплексированных магистралях после одного кода адреса передается несколько кодов данных (массив данных). Это позволяет существенно повысить быстродействие магистрали. Иногда в магистралях применяется частичное мультиплексирование, то есть часть разрядов данных передается по немultipлексированным линиям, а другая часть — по мультиплексированным с адресом линиям.

Шина управления — это вспомогательная шина, управляющие сигналы на которой определяют тип текущего цикла и фиксируют моменты времени, соответствующие разным частям или стадиям цикла. Кроме того, управляющие сигналы обеспечивают согласование работы процессора (или другого хозяина магистрали, задатчика, master) с работой памяти или устройства ввода/вывода (устройства-исполнителя, slave). Управляющие сигналы также обслуживают запрос и предоставление прерываний, запрос и предоставление прямого доступа.

Сигналы шины управления могут передаваться как в положительной логике, так и в отрицательной логике. Линии шины управления могут быть как однонаправленными, так и двунаправленными. Типы выходных каскадов могут быть самыми разными: с двумя состояниями (для однонаправленных линий), с тремя состояниями (для двунаправленных линий), с открытым коллектором (для двунаправленных и мультиплексированных линий).

Самые главные управляющие сигналы — это стробы обмена, то есть сигналы, формируемые процессором и определяющие моменты времени, в которые производится пересылка данных по шине данных, обмен данными. Чаще всего в магистрали используются два различных строба обмена:

- Строб записи (вывода), который определяет момент времени, когда устройство-исполнитель может принимать данные, выставленные процессором на шину данных ;
- Строб чтения (ввода), который определяет момент времени, когда устройство-исполнитель должно выдать на шину данных код данных, который будет прочитан процессором.

При этом большое значение имеет то, как процессор заканчивает обмен в пределах цикла, в какой момент он снимает свой строб обмена. Возможны два пути решения (рис. 25):

- При **синхронном обмене** процессор заканчивает обмен данными самостоятельно, через раз и навсегда установленный временной интервал выдержки ($t_{\text{выд}}$), то есть без учета интересов устройства-исполнителя;

- При **асинхронном обмене** процессор заканчивает обмен только тогда, когда устройство-исполнитель подтверждает выполнение операции специальным сигналом (так называемый режим handshake — рукопожатие).

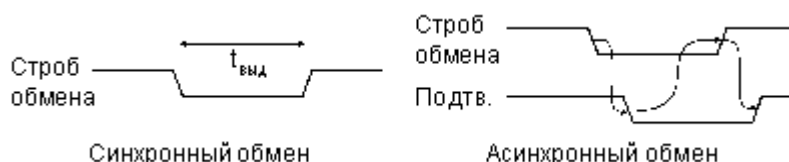


Рис. 25. Синхронный обмен и асинхронный обмен.

Достоинства синхронного обмена — более простой протокол обмена, меньшее количество управляющих сигналов. Недостатки — отсутствие гарантии, что исполнитель выполнил требуемую операцию, а также высокие требования к быстродействию исполнителя.

Достоинства асинхронного обмена — более надежная пересылка данных, возможность работы с самыми разными по быстродействию исполнителями. Недостаток — необходимость формирования сигнала подтверждения всеми исполнителями, то есть дополнительные аппаратурные затраты.

При асинхронном обмене требуется какое-то время на выработку, передачу дополнительного сигнала и на его обработку процессором. При синхронном обмене приходится искусственно увеличивать длительность строба обмена для соответствия требованиям большего числа исполнителей, чтобы они успевали обмениваться информацией в темпе процессора.

Поэтому иногда в магистрали предусматривают возможность как синхронного, так и асинхронного обмена, причем синхронный обмен является основным и довольно быстрым, а асинхронный применяется только для медленных исполнителей.

По используемому типу обмена магистрали микропроцессорных систем также делятся на синхронные и асинхронные.

2. ЦИКЛЫ ОБМЕНА ИНФОРМАЦИЕЙ

Циклы программного обмена

Рассмотрим для примера два довольно типичных случая программного обмена по магистрали микропроцессорной системы.

Первый пример — это обмен по мультиплексированной асинхронной магистрали Q-bus, предложенной фирмой DEC и широко применявшейся в микрокомпьютерах и промышленных контроллерах. Упрощенные временные диаграммы циклов чтения (ввода) и записи (вывода) по этой магистрали приведены на рис. 26 и 27.

Отметим, что в дальнейшем тексте знак "минус" перед названием сигнала говорит о том, что активный уровень сигнала низкий, пассивный — высокий, то есть сигнал отрицательный. Если минуса перед названием сигнала нет, то сигнал положительный, его низкий уровень пассивный, а высокий — активный.

На шине адреса / данных (AD) в начале цикла обмена (в фазе адреса) процессор (задатчик) выставляет код адреса. На этой шине используется отрицательная логика. Средний уровень сигналов на шине AD обозначает, что состояния сигналов на шине в данные временные интервалы не важны. Для стробирования адреса используется отрицательный синхросигнал -SYNC, выставляемый также процессором. Его передний (отрицательный) фронт соответствует действительности кода адреса на шине AD. Фаза адреса одинакова в обоих циклах записи и чтения.

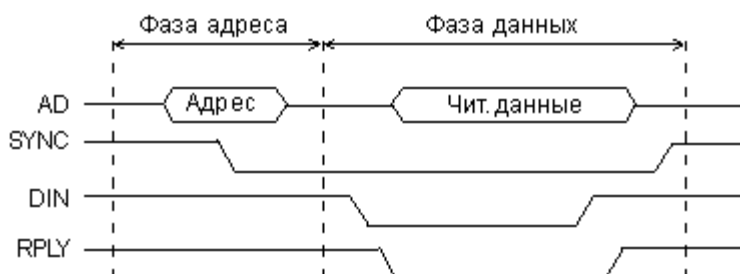


Рис. 26. Цикл чтения на магистрали Q-bus.

Получив (распознав) свой код адреса, устройство ввода/вывода или память (исполнитель) готовится к проведению обмена. Через некоторое время

после начала (отрицательного фронта) сигнала --SYNC процессор снимает адрес и начинает фазу данных.



Рис. 27. Цикл записи на магистрали Q-bus.

В фазе данных цикла чтения (рис. 2.3) процессор выставляет сигнал stroba чтения данных --DIN , в ответ на который устройство, к которому обращается процессор (исполнитель), должно выставить свой код данных (читаемые данные). Одновременно это устройство должно подтвердить выполнение операции сигналом подтверждения обмена --RPLY .

Для сигнала --RPLY используется тип выходного каскада ОК, чтобы не было конфликтов между устройствами-исполнителями. Процессор, получив сигнал --RPLY , заканчивает цикл обмена. Для этого он снимает сигнал --DIN и сигнал --SYNC . Устройство-исполнитель в ответ на снятие сигнала --DIN должно снять код данных с шины AD и закончить сигнал подтверждения --RPLY . После этого процессор снимает сигнал --SYNC .

В фазе данных цикла записи (рис. 2.4) процессор выставляет на шину AD код записываемых данных и сопровождает его отрицательным сигналом stroba записи данных --DOUT . Устройство-исполнитель должно по этому сигналу принять данные от процессора и сформировать сигнал подтверждения обмена --RPLY . Процессор, получив сигнал --RPLY , заканчивает цикл обмена. Для этого он снимает код данных с шины AD и сигнал --DOUT . Устройство-исполнитель в ответ на снятие сигнала --DOUT должно закончить сигнал подтверждения --RPLY . После этого процессор снимает сигнал --SYNC .

То есть на данной магистрали адрес передается синхронно (без подтверждения его получения исполнителем), а данные передаются

асинхронно, с обязательным подтверждением их выдачи или приема исполнителем. Отсутствие сигнала подтверждения –RPLY в течение заданного времени воспринимается процессором как аварийная ситуация. В принципе возможна и асинхронная передача адреса, что увеличивает надежность обмена, хотя может снижать его скорость.

Помимо циклов чтения и записи на магистрали Q-bus используются также и циклы типа "ввод-пауза-вывод" ("чтение-модификация-запись"). Упрощенная временная диаграмма этого цикла представлена на рис. 28.



Рис. 28. Цикл "ввод-пауза-вывод" на магистрали Q-bus.

В этом цикле адресная фаза производится точно так же, как и в циклах чтения (ввода) и записи (вывода). Но в фазе данных процессор производит сначала чтение из заданного в адресной фазе адреса, а потом запись в тот же самый адрес. Для чтения используется строб чтения -DIN, а для записи – строб записи -DOUT. В ответ на сигнал –DIN устройство-исполнитель выдает свои данные на шину AD, а по сигналу –DOUT – принимает данные с шины AD. Как и в циклах чтения и записи, устройство-исполнитель подтверждает выполнение каждой операции сигналом подтверждения -RPLY. Понятно, что цикл "ввод-пауза-вывод" требует больше времени, чем каждый из циклов чтения или записи, но меньше времени, чем два последовательно произведенных цикла чтения и записи (так как для него нужна только одна адресная фаза). Сигнал –SYNC вырабатывается процессором в начале цикла "ввод-пауза-вывод" и держится до окончания всего цикла.

В качестве второго примера рассмотрим циклы обмена на синхронной немультимплексированной магистрали ISA (Industrial Standard Architecture), предложенной фирмой IBM и широко используемой в персональных

компьютерах. Упрощенные циклы записи в устройство ввода/вывода и чтения из устройства ввода/вывода приведены на рис. 29 и 30.

Оба цикла начинаются с выставления процессором (здатчиком) кода адреса на шину адреса SA (логика на этой шине положительная). Адрес остается на шине SA до конца цикла. Фаза адреса, одинаковая для обоих циклов, заканчивается с началом строба обмена данными –IOR или -IOW. В течение фазы адреса устройство-исполнитель должно принять код адреса и распознать или не распознать его. Если адрес распознан, исполнитель готовится к обмену.

В фазе данных цикла чтения (рис. 29) процессор выставляет отрицательный сигнал чтения данных из устройства ввода/вывода -IOR. В ответ на него устройство-исполнитель должно выдать на шину данных SD свой код данных (читаемые данные). Логика на шине данных положительная. Через установленное время строб обмена –IOR снимается процессором, после чего снимается также и код адреса с шины SA. Цикл заканчивается без учета быстродействия исполнителя.



Рис. 29. Цикл чтения из УВВ на магистрали ISA.



Рис. 30 Цикл записи в УВВ на магистрали ISA.

Но так происходит только в случае основного, синхронного обмена. Кроме него на магистрали ISA также предусмотрена возможность

асинхронного обмена. Для этого применяется сигнал готовности канала (магистралей) I/O CH RDY. Тип выходного каскада для данного сигнала — ОК, для предотвращения конфликтов между устройствами-исполнителями. При синхронном обмене сигнал I/O CH RDY всегда положительный. Но медленное устройство-исполнитель, не успевающее работать в темпе процессора, может этот сигнал снять, то есть сделать нулевым сразу после начала строба обмена. Тогда процессор до того момента, пока сигнал I/O CH RDY не станет снова положительным, приостанавливает завершение цикла, продлевает строб обмена. Конечно, слишком большая длительность этого сигнала рассматривается как аварийная ситуация. Для простоты понимания можно считать, что устройство-исполнитель формирует в данном случае отрицательный сигнал неготовности завершить обмен. На время этого сигнала обмен на магистрали приостанавливается.

Принципиальное отличие асинхронного обмена по магистрали ISA от асинхронного обмена по магистрали Q-bus состоит в следующем. Если в случае Q-bus сигнал подтверждения обязателен, и его должен формировать каждый исполнитель, то в случае ISA сигнал о неготовности исполнитель может не формировать, если он успевает работать в темпе процессора. Зато в случае Q-bus к концу цикла обмена процессор всегда уверен, что устройство-исполнитель выполнило требуемую операцию, а в случае ISA такой уверенности нет.

В фазе данных цикла записи по магистрали ISA (рис. 2.7) процессор выставляет на шину данных SD код записываемых данных и сопровождает их стробом записи данных в устройство ввода/вывода -IOW. Получив этот сигнал, устройство-исполнитель должно принять с шины SD код записываемых данных. Если оно не успевает сделать это в темпе процессора, то оно может снять на нужное время сигнал I/O CH RDY после получения переднего фронта сигнала -IOW. Тогда процессор приостановит окончание цикла записи.

Рассмотренные примеры, конечно, не раскрывают всех тонкостей обмена по упомянутым магистралям. Они всего лишь иллюстрируют главные принципы обмена по ним.

Циклы обмена по прерываниям

Циклы обмена в режиме прерываний строятся по тем же принципам, что и циклы программного обмена, но имеют ряд специфических особенностей.

Прерывания в микропроцессорных системах бывают двух основных типов:

- векторные прерывания, которые требуют проведения цикла чтения по магистрали;
- радиальные прерывания, которые не требуют никакого цикла обмена по магистрали.

Дело в том, что прерываний в микропроцессорной системе обычно бывает много. Поэтому процессору необходима информация о номере (или, как еще говорят, об адресе вектора) конкретного прерывания. Эта информация может быть передана процессору двумя путями.

При **векторном прерывании** код номера прерывания передается процессору тем устройством ввода/вывода, которое данное прерывание запросило. Для этого процессор проводит цикл чтения по магистрали, и по шине данных получает код номера прерывания. Шина адреса в данном цикле обычно не используется, так как устройство, запросившее прерывание, и так знает, что процессор будет обращаться именно к нему. В этом случае в магистрали достаточно всего одной линии запроса прерывания для всех устройств ввода/вывода. Так организованы прерывания, например, в магистрали Q-bus.

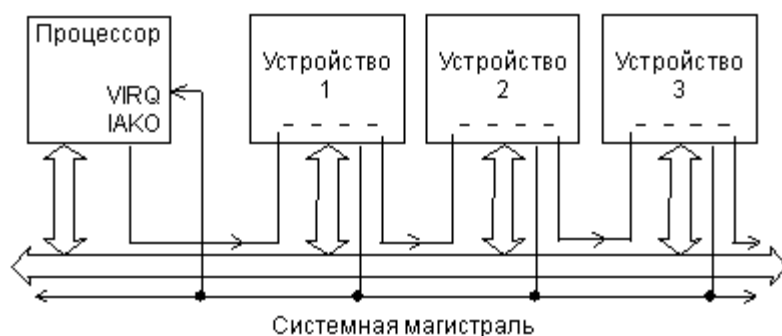


Рис. 31. Сигналы запроса и предоставления прерывания в магистрали Q-bus.

Схема распространения сигналов, участвующих в прерываниях на магистрали Q-bus, показана на рис. 31. Упрощенная временная диаграмма цикла запроса и предоставления магистрали представлена на рис. 32.

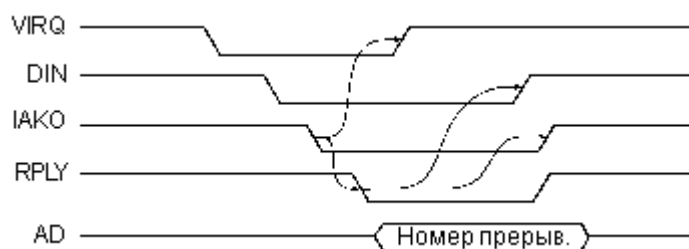


Рис. 32. Цикл запроса/предоставления векторного прерывания на магистрали Q-bus.

Запрос прерывания осуществляется отрицательным сигналом -VIRQ, который может формироваться каждым из устройств, запрашивающих прерывание. Тип выходного каскада для этого сигнала — ОК, чтобы избежать конфликтов между запрашивающими прерывания устройствами. Получив сигнал -VIRQ, процессор предоставляет прерывание (закончив предварительно выполнение текущей команды). Для этого он выставляет сигнал чтения данных –DIN и сигнал предоставления прерывания IAKO. Этот сигнал IAKO последовательно проходит через все устройства, которые могут запрашивать прерывания. Если устройство запросило прерывание, то оно не пропускает через себя этот сигнал. В результате получается, что если прерывания одновременно запросили два или более устройств, то сигнал предоставления прерывания получит только одно устройство, а именно то,

которое ближе к процессору. Такой механизм разрешения конфликтов называется иногда географическим приоритетом (или цепочечным приоритетом, Daisy Chain). Получив сигнал IAKO, устройство, запросившее прерывание, должно снять свой сигнал -VIRQ.

Затем процессор проводит цикл безадресного чтения номера прерывания. В ответ на полученные сигналы -DIN и IAKO устройство, которому предоставлено прерывание, должно выдать на шину адреса / данных AD код номера прерывания (адреса вектора прерывания) и выставить сигнал подтверждения -RPLY. Процессор читает код номера прерывания и заканчивает цикл безадресного чтения снятием сигналов -DIN и IAKO.

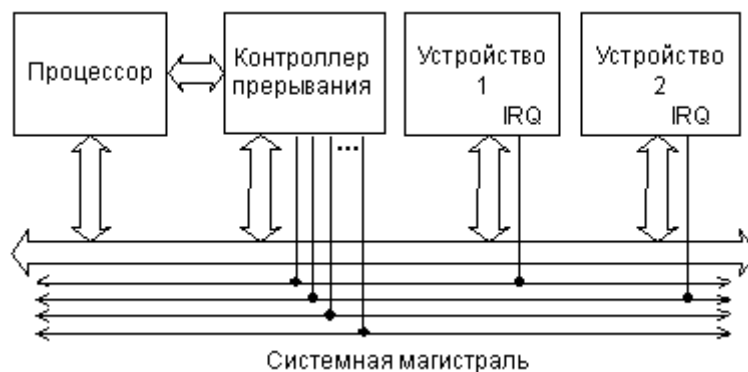


Рис. 33. Структура связей для организации радиальных прерываний на магистрали ISA.

При **радиальном прерывании** в магистрали имеется столько линий запроса прерывания, сколько всего может быть разных прерываний. То есть каждое устройство ввода/вывода, желающее использовать прерывание, подает сигнал запроса прерывания по своей отдельной линии. Процессор узнает о номере прерывания по номеру линии, по которой пришел сигнал запроса прерывания. Никаких циклов обмена по магистрали при этом не требуется. В случае радиальных прерываний в систему обычно включается дополнительная микросхема контроллера прерываний, обрабатывающая сигналы запросов прерываний. Именно так организованы прерывания, например, в магистрали ISA.

Упрощенная структура связей между устройствами, участвующими в обмене по прерываниям, на магистрали ISA показана на рис. 33. Процессор общается с контроллером прерываний как по магистрали (чтобы задать ему режимы работы), так и вне магистрали (при обработке запросов на прерывание). Сигналы запросов прерываний IRQ распределяются между всеми устройствами магистрали. На каждую линию IRQ приходится одно устройство. Тип выходного каскада для этих линий — 2С, так как конфликты здесь не предусмотрены. Запросом прерывания является передний, положительный фронт сигнала IRQ. При одновременном поступлении сигналов IRQ от нескольких устройств порядок их обслуживания определяется контроллером прерываний.

Векторное и радиальное прерывание

Векторные прерывания обеспечивают системе большую гибкость, в системе их может быть очень много. Но зато они требуют дополнительных аппаратурных узлов во всех устройствах, запрашивающих прерывания, для обслуживания циклов безадресного чтения.

Радиальных прерываний в системе обычно не очень много (от 1 до 16). При этом типе прерываний, как правило, требуется введение в систему специального контроллера прерываний. Каждое радиальное прерывание требует введения дополнительной линии в шину управления системной магистрали. Но работать с радиальными прерываниями проще, так как все сводится только к выработке единственного сигнала IRQ, и никаких циклов обмена по магистрали не требуется.

Циклы обмена в режиме ПДП

Циклы обмена в режиме прямого доступа к памяти выполняются по тем же правилам, что и циклы программного обмена, и циклы предоставления прерываний.

Прежде чем начать обмен в режиме ПДП, устройство, которому необходим ПДП, должно запросить ПДП и получить его. Процедура запроса

и предоставления ПДП очень похожа на процедуру запроса и предоставления прерывания. В обоих случаях устройство, требующее обслуживания, посылает сигнал запроса процессору. Однако в случае ПДП процессор обязательно должен предоставить ПДП запросившему устройству с помощью специальных сигналов, так как на время ПДП процессор отключается от магистрали. А при радиальных прерываниях предоставления прерывания от процессора не требуется.

На магистрали Q-bus запрос и предоставление ПДП организуются подобно запросу и предоставлению прерывания. Упрощенная структура связей устройств, участвующих в ПДП, показана на рис. 34. Временная диаграмма запроса/предоставления ПДП очень близка к временной диаграмме запроса/предоставления прерывания (см. рис. 32).

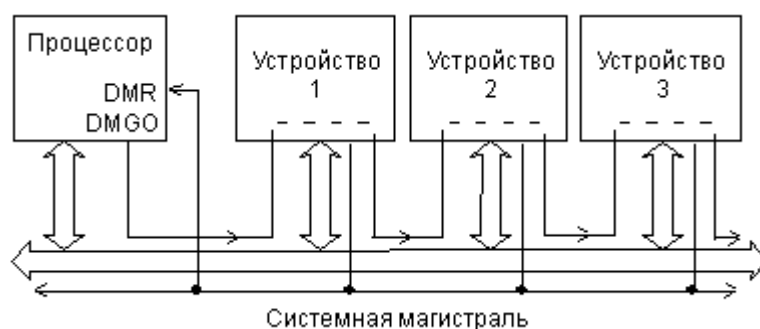


Рис. 34. Структура связей запроса/предоставления ПДП на магистрали Q-bus.

Сигнал запроса ПДП, называемый -DMR, передается всеми устройствами, нуждающимися в ПДП, по одной линии магистрали. Тип выходного каскада на этой линии — ОК. Процессор, получив сигнал -DMR, выдает сигнал предоставления ПДП DMGO, аналогичный сигналу IAKO. Этот сигнал также проходит через все устройства последовательно, в результате чего ПДП получает только то устройство, которое находится ближе к процессору (географический приоритет). А затем устройство, получившее ПДП, проводит циклы обмена по магистрали, аналогично циклам программного обмена. В циклах ПДП информация читается из памяти и

записывается в устройство ввода/вывода, или наоборот — читается из устройства ввода/вывода и передается в память.

На магистрали ISA запрос/предоставление ПДП очень напоминает организацию радиальных прерываний (рис. 35). Точно так же в системе существует контроллер ПДП, к которому сходятся сигналы запроса ПДП, называемые DRQ, и от которого расходятся сигналы предоставления ПДП, называемые -DACK. К каждому каналу ПДП (пара сигналов DRQ и -DACK) подключается только одно устройство, запрашивающее ПДП. Тип выходных каскадов для этих сигналов — 2С. Устройство, нуждающееся в ПДП, посылает сигнал запроса DRQ и получает в ответ сигнал предоставления -DACK. После этого контроллер ПДП проводит циклы обмена по магистрали между устройством ввода/вывода и памятью.

Упрощенная временная диаграмма циклов ПДП на магистрали ISA показана на рис. 36.

На магистрали ISA используются отдельные стробы записи в память (-MEMW) и записи в устройства ввода/вывода (-IOW), а также отдельные стробы чтения из памяти (-MEMR) и чтения из устройств ввода/вывода (-IOR). Это позволяет за один цикл обмена ПДП читать информацию из памяти и записывать ее в устройство ввода/вывода или же читать информацию из устройства ввода/вывода и записывать ее в память. При этом на шине адреса выставляется адрес памяти, а адрес устройства ввода/вывода заменяется одним-единственным сигналом AEN. Естественно, в цикле обмена в режиме ПДП участвует только то устройство ввода/вывода, которое предварительно запросило ПДП и которому ПДП было предоставлено. Поэтому никаких конфликтов между устройствами ввода/вывода из-за такой упрощенной адресации не возникает.



Рис. 35. Структура связей запроса/предоставления ПДП на магистрали ISA.



Рис. 36. Цикл ПДП на магистрали ISA.

Прохождение сигналов по магистрали

При организации обмена по магистралям и шинам разработчику необходимо учитывать несколько важных моментов, связанных как с особенностью распространения сигналов по шинам, так и с самой природой шин. В противном случае микропроцессорная система может попросту не работать или работать неустойчиво, хотя вся логика цифровых устройств, входящих в систему, будет спроектирована безошибочно.

В случае, когда системная шина (магистраль) микропроцессорной системы является внешней, а не скрыта внутри микросхемы, необходимо учитывать особенности распространения сигналов по длинным линиям. Хотя в большинстве случаев длина магистрали не слишком велика, не превышает 1—2 десятков сантиметров, это все равно оказывает большое влияние на синхронизацию обмена.

На прохождение сигналов по магистрали влияют следующие факторы:

- конечная величина задержки распространения сигналов по линиям магистрали;
- различие задержек распространения сигналов по разным линиям шины;
- неодновременное выставление сигналов на линии шины;
- искажение фронтов сигналов, проходящих по линиям магистрали;
- отражение сигналов от концов линий связи (рис. 37).



Рис. 37. Прохождение сигналов по шине.

Для учета всех этих факторов разработчики стандартных магистралей обмена и стандартных протоколов обмена всегда закладывают необходимые задержки между сигналами, участвующими в обмене. Кроме того, задержки между сигналами выбираются таким образом, чтобы устройство, которому адресован тот или иной сигнал, имело достаточно времени для его обработки. Если разрабатывается новая магистраль, все это тоже надо учитывать.

Поэтому пытаться "модернизировать" какой-то стандартный протокол и ускорять обмен по магистрали путем уменьшения задержек, предусмотренных стандартом, очень опасно. Точно так же опасно, не изменяя протокола обмена, пытаться увеличить длину магистрали, увеличивая тем самым задержки распространения сигналов по линиям и шинам. Особенно чувствительны к такого рода "модернизациям" синхронные магистрали, в которых не предусмотрено обязательное подтверждение выполнения каждой операции.

Например, длительность фазы адреса в цикле обмена выбирается таким образом. В течение адресной фазы все сигналы всех разрядов кода адреса, пусть даже и сформированные процессором не одновременно, должны прийти

до устройства-исполнителя по своим проводам шины. А устройство-исполнитель должно этот код адреса принять и обработать (то есть отличить свой адрес от чужого). Естественно, для гарантии в длительность адресной фазы еще добавляется небольшая дополнительная задержка.

Точно так же длительность фазы данных в цикле чтения должна выбираться такой, чтобы устройство-исполнитель успело получить строб чтения и выдать код читаемых данных на шину данных. Затем этот код должен успеть дойти до процессора и процессор должен успеть его прочесть. После чего процессор снимает сигнал строга чтения, этот задний фронт сигнала доходит с задержкой до устройства-исполнителя, которое также с задержкой снимает свой код данных. Аналогично и в цикле записи.

Для улучшения формы сигналов, распространяющихся по магистрали, иногда применяют оконечные согласователи (терминаторы) на концах линий магистрали. Особенно важно их применение в случае, когда допустимая длина магистрали превышает несколько метров. Например, в случае магистрали Q-bus применяются два типа согласователей: 120-омный и 250-омный (рис. 38).

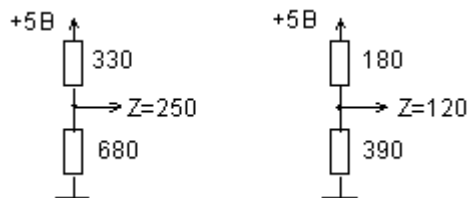


Рис. 38. Оконечные согласователи на магистрали Q-bus.

Включение согласователей предъявляет дополнительные требования к нагрузочной способности передатчиков, работающих на линии магистрали. В магистрали ISA подобные согласователи не используются, хотя к некоторым линиям присоединены резисторы, соединенные другим своим выводом с шиной питания (прежде всего это линии, тип выходного каскада для которых — ОК).

В любом случае выходные каскады передатчиков, работающих на линии магистрали, должны обеспечивать высокие выходные токи, так как к

магистральной может подключаться несколько устройств, каждое из которых потребляет входной ток. Типичные величины требуемых выходных токов магистральных передатчиков находятся в пределах 20—30 мА. В то же время входные токи магистральных приемников должны быть малыми, чтобы не перегружать передатчики. Типичные величины допустимых входных токов магистральных приемников лежат в пределах 0,2—0,8 мА.

ОРГАНИЗАЦИЯ ОБМЕНА ИНФОРМАЦИЕЙ

Микропроцессор взаимодействуют с внешними устройствами с помощью периферийных устройств. Поэтому необходимо установить единые правила обмена информацией и требования к средствам сопряжения МП с остальными устройствами системы. Эти цели реализует унифицированный интерфейс.

Под интерфейсом понимают совокупность технических средств и правил, устанавливающих единые принципы взаимодействия устройств системы (протокол обмена). В состав интерфейса входят:

- аппаратные средства (разъемы, линии связи, согласующие устройства)
- номенклатуры и характер связей
- программные средства
- описание электрофизических параметров сигналов.

Для интерфейса характерно выполнение четырех функций:

1. буферизация информации (для согласования и синхронизации процессора и периферийных устройств)
2. дешифровка адреса (для выбора конкретного устройства в системе)
3. дешифровка команды (производится для устройств, которые кроме передачи данных выполняют какие-либо действия: переключение диапазона АЦП, перемотка ленты, перемещение головок дисковых накопителей и т.д.),
4. синхронизация и управление процессом обмена (требуется при реализации любой из перечисленных функций)

В микропроцессорных системах наиболее часто используется магистральный интерфейс (интерфейсы с разветвленными связями дороже и сложнее в реализации). Единая информационная магистраль связывает между собой все устройства микропроцессорной системы и функционально состоит из информационных шин адреса, данных, сигналов управления (команд). Две или все три шины бывают объединены в одну группу шин при

реализации обмена информацией в режиме разделения времени (мультиплексированная магистраль). Шина адреса в простых МП системах является однонаправленной, т.к. в простейших системах только МП может вырабатывать сигналы кода адреса, а остальные устройства, подключенные к ША непрерывно выполняют операцию приема и опознавания кода адреса. В сложных или мультиплексорных системах ША должна быть двунаправленной для части или всех ФБ системы.

Часть устройств системы по ШД может работать в двунаправленном режиме (МП, ОЗУ). Часть устройств может работать только на прием информации (устройство печати), другие только передавать информацию (ПЗУ). Разрядность ШД определяется разрядностью МП.

Организация взаимодействия и обмена информацией между устройствами (модулями) МП - системы должна обеспечивать:

- включение в систему новых устройств без каких-либо переделок в аппаратуре, а лишь путем добавления программ, обслуживающих эти устройства, возможность реализовать микропроцессорные системы с различной конфигурацией;

- возможность эффективной реализации обмена информацией в системе, содержащей устройства со значительно различающимися скоростями передачи данных, причем в условиях, когда запросы на операции ввода – вывода поступают в произвольные моменты времени (асинхронно относительно программы, выполняемой процессором) и имеют разную срочность исполнения;

- возможность параллельного во времени выполнения процессором программы, а периферийными устройствами операций ввода – вывода;

- унификацию программирования операций ввода – вывода с исключением крайне важности учета особенностей того или иного периферийного устройства.

Указанные требования реализуются на базе следующих архитектурных решений, характерных для построения микропроцессорных средств.

Используется магистрально-модульная организация микропроцессорных систем – отдельные микропроцессорные средства выполняются в виде конструктивно законченных модулей (БИС), и эти модули объединяются в систему посредством общих шин (магистралей). Особенностью архитектуры микропроцессорных средств является связь системы прерывания с шинами и процедурами работы интерфейса.

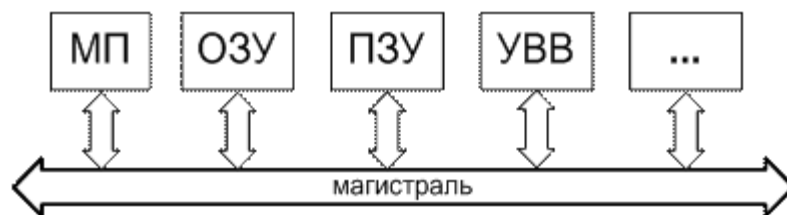


Рис.39. Магистрально-модульная структура МП - системы

Магистрально-модульная структура МП - системы.

Наличие нескольких способов (режимов) передачи информации между устройствами через интерфейс повышает гибкость интерфейса, позволяя выбрать наиболее подходящий режим с учетом характеристик ПУ и структуры передаваемого сообщения (отдельное слово или массив слов).

Важной особенностью организации обмена информацией в микропроцессорных системах является использование специализированных интерфейсных БИС (контроллер прямого доступа, контроллер прерываний, программируемый периферийный адаптер, программируемый связной адаптер, программируемый таймер), позволяющих в значительной степени освободить процессор от управления операциями ввода – вывода и выполнения вспомогательных процедур преобразования форматов данных, подсчета передаваемых байт и др.

Программная настраиваемость интерфейсных БИС обеспечивает построение гибких и эффективных микропроцессорных систем управления и обработки данных.

В микропроцессорных системах используются три способа организации передачи информации:

- 1) программно-управляемая передача, инициируемая процессором;
- 2) программно-управляемая передача, инициируемая запросом прерывания от периферийного устройства;
- 3) прямой доступ к памяти (ПДП или DMA).

При **программно-управляемой передаче** обмен данными происходит через регистры процессора и производится соответствующими командами программы процессора. При этом способе передача инициируется самим процессором (точнее соответствующей командой в его программе).

Передача по прерыванию инициируется запросом прерывания от периферийного устройства.

Процедура передачи по прерыванию имеет ряд особенностей. При программно-управляемой передаче данных микропроцессор на все время этой операции отвлекается от выполнения основной программы, что ведет к снижению производительности МП – системы. Особенно это становится заметным при пересылке блока данных, когда микропроцессор для каждого передаваемого байта должен выполнить много команд, чтобы обеспечить буферизацию данных, подсчет количества переданных байт, формирование адресов в памяти и т.д. Более того, скорость передачи данных через микропроцессор может оказаться недостаточной для работы с высокоскоростными ПУ.

Для быстрого ввода – вывода блоков данных и разгрузки микропроцессоров от управления операциями ввода – вывода используют прямой доступ к памяти.

Прямой доступ к памяти (ПДП) принято называть способ обмена данными, обеспечивающий автономно от микропроцессора (ядром) установление связи и передачу данных между ОЗУ и внешним устройством (ВУ). Прямой доступ к памяти повышает предельную скорость ввода – вывода информации и делает МП- систему более приспособленной для

работы в системах реального времени. Прямым доступом к памяти управляет контроллер ПДП, выполняющий следующие функции: управление иницируемой процессором или ВУ передачей данных между ОЗУ и ВУ; задание размера блока данных, который подлежит передаче, и области памяти, используемой при передаче; формирование адресов ячеек ОЗУ, участвующих в передаче; подсчет числа байт, передаваемых через интерфейс, определение момента завершения заданной операции ввода – вывода.

ФОРМАТЫ ПЕРЕДАЧИ ДАННЫХ

Существуют два способа передачи слов информации по линиям данных: параллельный, когда одновременно пересылаются все биты слова, и последовательный, когда биты слова пересылаются поочередно, начиная, например, с его младшего разряда.

Так как между отдельными проводниками шины для параллельной передачи данных существует электрическая емкость, то при изменении сигнала, передаваемого по одному из проводников, возникает помеха (короткий выброс напряжения) на других проводниках. С увеличением длины шины (увеличением емкости проводников) помехи возрастают и могут восприниматься приемником как сигналы. Поэтому рабочее расстояние для шины параллельной передачи данных ограничивается длиной 1-2 м, и только за счет существенного удорожания шины или снижения скорости передачи длину шины можно увеличить до 10-20 м.

Возможны два режима последовательной передачи данных: синхронный и асинхронный.

При синхронной последовательной передаче каждый передаваемый бит данных сопровождается импульсом синхронизации, информирующим приемник о наличии на линии информационного бита. Следовательно, между передатчиком и приемником должны быть протянуты минимум три провода: два для передачи импульсов синхронизации и бит данных, а также

общий заземленный проводник. Если же передатчик (например, микроЭВМ) и приемник (например, дисплей) разнесены на несколько метров, то каждый из сигналов (информационный и синхронизирующий) придется посылать либо по экранированному (телевизионному) кабелю, либо с помощью витой пары проводов, один из которых заземлен или передает сигнал, инверсный основному.

Синхронная последовательная передача начинается с пересылки в приемник одного или двух символов синхронизации (не путать с импульсами синхронизации). Получив такой символ (символы), приемник начинает прием данных и их преобразование в параллельный формат. Естественным, что при такой организации синхронной последовательной передачи она целесообразна лишь для пересылки массивов слов, а не отдельных символов. Это обстоятельство, а также необходимость использования для обмена сравнительно дорогих (четырехпроводных или кабельных) линий связи помешало широкому распространению синхронной последовательности передачи данных.

Асинхронная последовательная передача данных означает, что у передатчика и приемника нет общего генератора синхроимпульсов и что синхронизирующий сигнал не посылается вместе с данными. Как же в таком случае приемник будет узнавать о моментах начала и завершения передачи бит данных. Опишем простую процедуру, которую можно использовать, если передатчик и приемник асинхронной последовательной передачи данных согласованы по формату и скорости передачи.

Стандартный формат асинхронной последовательной передачи данных, содержит n пересылаемых бит информации (при пересылке символов n равно 7 или 8 битам) и 3-4 дополнительных бита: стартовый бит, бит контроля четности (или нечетности) и 1 или 2 стоповых бита (рис. 40,а). Бит четности (или нечетности) может отсутствовать. Когда передатчик бездействует (данные не посылаются на линию), на линии сохраняется уровень сигнала, соответствующий логической 1.



Рис. 40. Формат асинхронной последовательной передачи данных

Передатчик может начать пересылку символа в любой момент времени посредством генерирования стартового бита, т. е. перевода линии в состояние логического 0 на время, точно равное времени передачи бита. Затем происходит передача битов символа, начиная с младшего значащего бита, за которым следует дополнительный бит контроля по четности или нечетности. Далее с помощью стопового бита линия переводится в состояние логической 1 (рис. 40,б). При единичном бите контроля стоповый бит не изменяет состояния сигнала на линии. Состояние логической 1 должно поддерживаться в течение промежутка времени, равного 1 или 2 временам передачи бита.

Промежуток времени от начала стартового бита до конца стопового бита (стоповых бит) называется кадром. Сразу после стоповых бит передатчик может посылать новый стартовый бит, если имеется другой символ для передачи; в противном случае уровень логической 1 может сохраняться на протяжении всего времени, пока бездействует передатчик. Новый стартовый бит может быть послан в любой момент времени после окончания стопового бита, например, через промежуток времени, равный 0.43 или 1.5 времени передачи бита.

В линиях последовательной передачи данных передатчик и приемник должны быть согласованы по всем параметрам формата, включая

номинальное время передачи бита. Для этого в приемнике устанавливается генератор синхроимпульсов, частота которого должна совпадать с частотой аналогичного генератора передатчика. Кроме того, для обеспечения оптимальной защищенности сигнала от искажения, шумов и разброса частоты синхроимпульсов приемник должен считывать принимаемый бит в середине его длительности. Рассмотрим работу приемника с того момента, когда он закончил прием символа данных и перешел в режим обнаружения стартового бита следующего слова.

Если линия перешла в состояние логического нуля и находится в этом состоянии в течение времени, не меньшего половины временного интервала передачи бита, то приемник переводится в режим считывания бит информации. В противном случае приемник остается в режиме обнаружения, так как вероятнее всего это был не стартовый бит, а шумовая помеха. В новом режиме приемник вырабатывает сигналы считывания через интервалы, равные времени передачи бита, т. е. выполняет считывание и сохранение принимаемых бит примерно на середине их передачи. Аналогичным образом будут считаны бит контроля четности и сигнал логической единицы (стоповый бит). Если оказалось, что на месте стопового бита обнаружен сигнал логического нуля, то произошла "Ошибка кадра" и символ принят неправильно. Иначе проверяется, четно ли общее число единиц в информационных битах и бите контроля, и если оно четно, производится запись принятого символа в буфер приемника.

Передний фронт стартового бита сигнализирует о начале поступления передаваемой информации, а момент его появления служит точкой отсчета времени для считывания бит данных. Стоповый бит предоставляет время для записи принятого символа в буфер приемника и обеспечивает возможность выявления ошибки кадра. Наиболее часто ошибки кадра появляются тогда, когда приемник ошибочно синхронизирован с битом 0, который в действительности не является стартовым битом. Если передатчик бездействует (посылает сигнал логической единицы) в течение одного кадра

или более, то всегда можно восстановить правильную синхронизацию. Хуже обстоит дело при рассинхронизации генераторов передатчика и приемника, когда временной интервал между сигналами считывания принимаемых битов будет меньше или больше времени передачи бита.

Например, если при считывании битов послышки, показанной на рис. 23 б, временной интервал между сигналами считывания станет на 6% меньше, чем время передачи бита, то восьмой и девятый сигналы считывания будут выработаны тогда, когда на линии находится бит контроля четности (рис. 22). Следовательно, не будет обнаружен стоповый бит и будет зафиксирована ошибка кадра, несмотря на правильность принятой информации. Однако при 18%-й рассинхронизации генераторов, когда вместо кода (01110001) приемник зафиксирует код (11100001), никаких ошибок не будет обнаружено - четность соблюдена и стоповый (девятый по порядку) бит равен 1 (см. рис. 41).

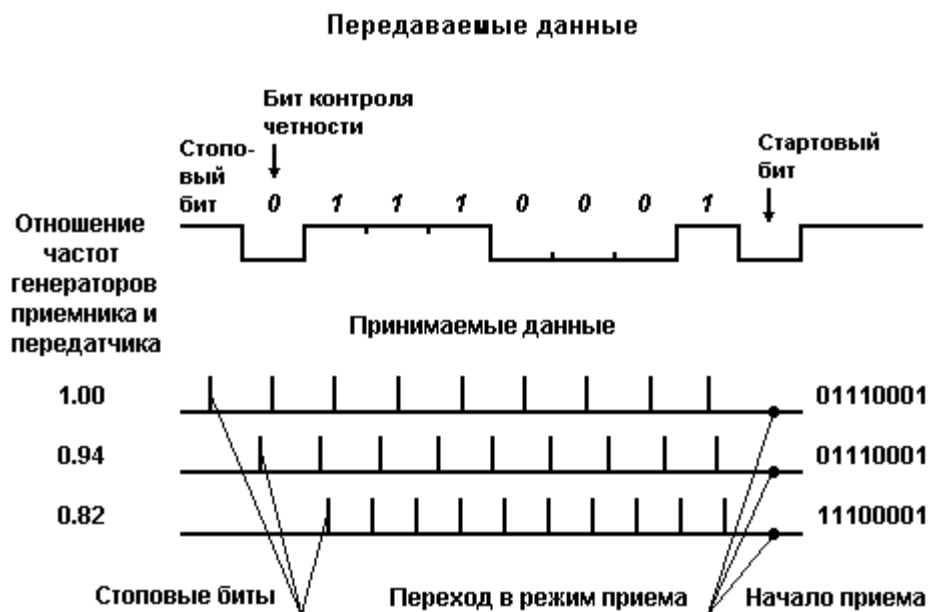


Рис. 41. Ошибка из-за рассинхронизации генераторов передатчика и приемника

4.1 ПАРАЛЛЕЛЬНАЯ ПЕРЕДАЧА ДАННЫХ

Параллельная передача данных между контроллером и ВУ является по своей организации наиболее простым способом обмена. Для организации параллельной передачи данных помимо шины данных, количество линий в которой равно числу одновременно передаваемых битов данных, используется минимальное количество управляющих сигналов.

В простом контроллере внешнего устройства, обеспечивающем побайтную передачу данных на внешнее устройство (рис. 42), в шине связи с внешним устройством используются всего два управляющих сигнала: "Выходные данные готовы" и "Данные приняты".

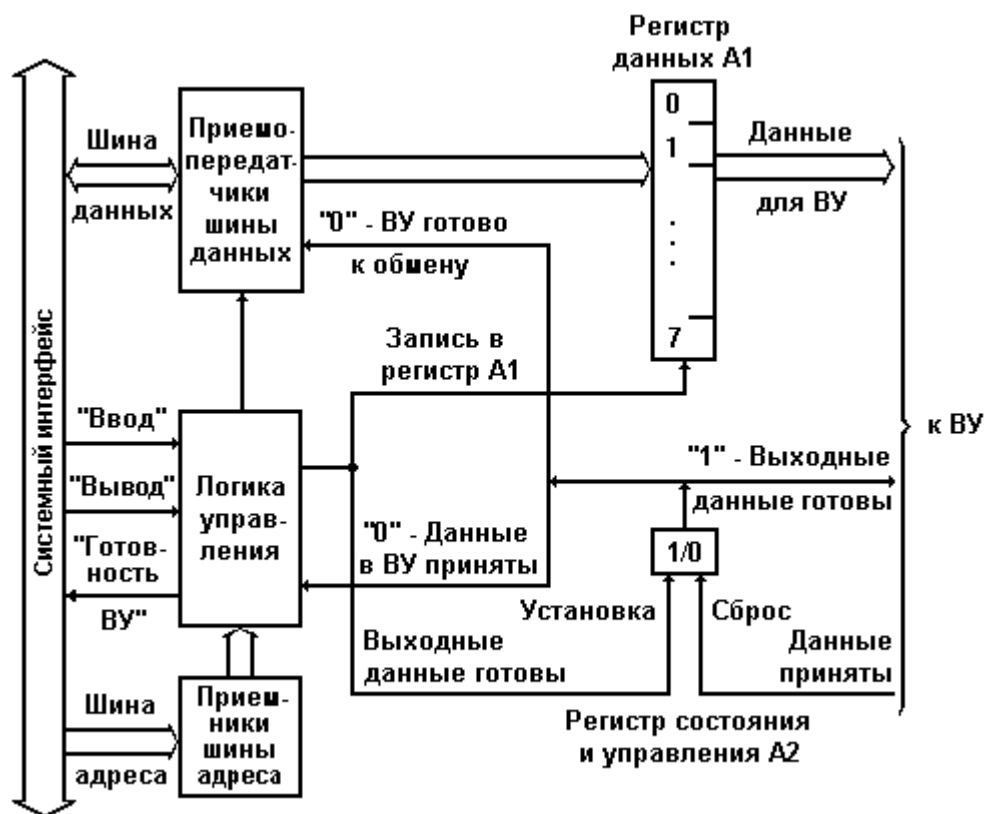


Рис. 42. Простой параллельный контроллер вывода.

Для формирования управляющего сигнала "Выходные данные готовы" и приема с внешнего устройства управляющего сигнала "Данные приняты" в контроллере используется одноразрядный адресуемый регистр состояния и управления A2 (обычно используются раздельные регистр состояния и регистр управления). Одновременно с записью очередного байта данных с

шины данных системного интерфейса в адресуемый регистр данных контроллера (порт вывода A1) в регистр состояния и управления записывается логическая единица. Тем самым формируется управляющий сигнал "Выходные данные готовы" в шине связи с внешним устройством.

Внешнее устройство, приняв байт данных, управляющим сигналом "Данные приняты" обнуляет регистр состояния контроллера. При этом формируются управляющий сигнал системного интерфейса "Готовность внешнего устройства" и признак готовности внешнего устройства к обмену, передаваемый в процессор по одной из линий шины данных системного интерфейса посредством стандартной операции ввода при реализации программы асинхронного обмена.

Логика управления контроллера обеспечивает селекцию адресов регистров контроллера, прием управляющих сигналов системного интерфейса и формирование на их основе внутренних управляющих сигналов контроллера, формирование управляющего сигнала системного интерфейса "Готовность внешнего устройства". Для сопряжения регистров контроллера с шинами адреса и данных системного интерфейса в контроллере используются соответственно приемники шины адреса и приемопередатчики шины данных.

Рассмотрим на примере, каким образом контроллер внешнего устройства обеспечивает параллельную передачу данных на внешнее устройство под управлением программы асинхронного обмена. Алгоритм асинхронного обмена в данном случае передачи прост

1. Микропроцессор проверяет готовность внешнего устройства к приему данных
2. Если внешнее устройство готово к приему данных (в данном случае это логический 0 в нулевом разряде регистра A2), то данные передаются с шины данных системного интерфейса в регистр данных A1 контроллера и далее на внешнее устройство. Иначе повторяется п. 1.

Блок-схема простого контроллера внешнего устройства, обеспечивающего побайтный прием данных с внешнего устройства, приведена на рис.43. В этом контроллере при взаимодействии с внешним устройством также используются два управляющих сигнала: "Данные от внешнего устройства готовы" и "Данные приняты".

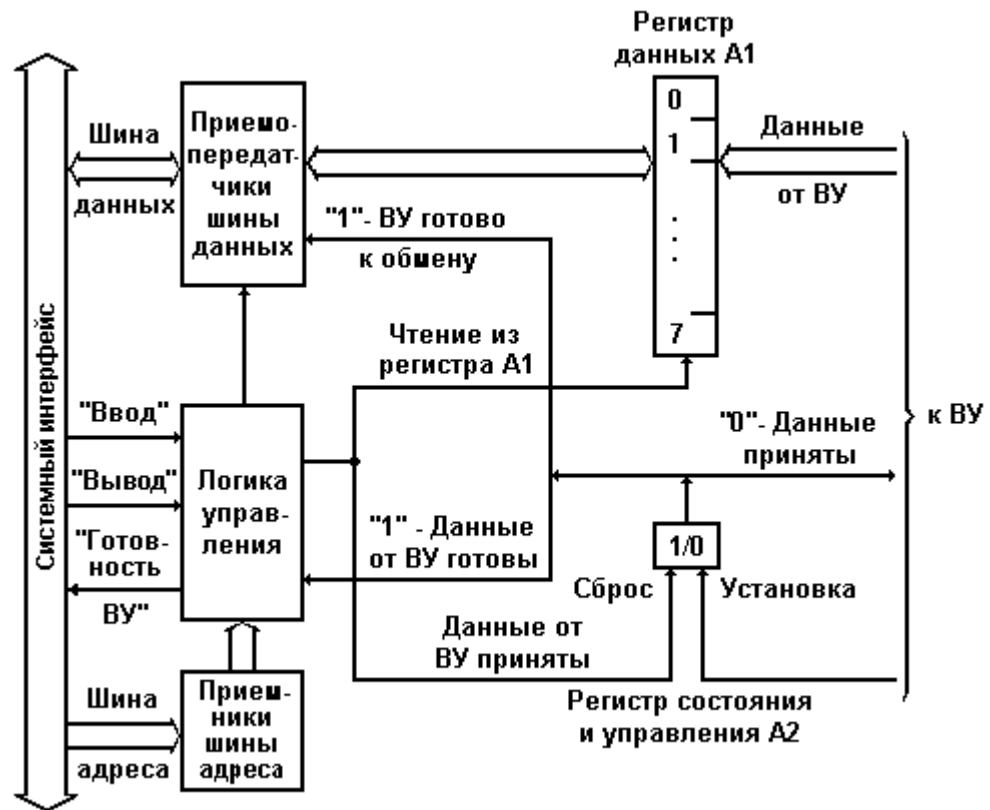


Рис. 43. Простой параллельный контроллер ввода

Для формирования управляющего сигнала "Данные приняты" и приема с внешнего устройства управляющего сигнала "Данные от внешнего устройства готовы" используется одноразрядный адресуемый регистр состояния и управления A2.

Внешнее устройство записывает в регистр данных контроллера A1 очередной байт данных и управляющим сигналом "Данные от внешнего устройства готовы" устанавливает в единицу регистр состояния и управления A2.

При этом формируются: управляющий сигнал системного интерфейса "Готовность внешнего устройства"; признак готовности внешнего

устройства к обмену, передаваемый в процессор по одной из линий шины данных системного интерфейса посредством операции ввода при реализации программы асинхронного обмена.

Тем самым контроллер извещает процессор о готовности данных в регистре A1. Процессор, выполняя программу асинхронного обмена, читает байт данных из регистра данных контроллера и обнуляет регистр состояния и управления A2. При этом формируется управляющий сигнал "Данные приняты" в шине связи с внешним устройством.

Логика управления контроллера и приемопередатчики шин системного интерфейса выполняют те же функции, что и в контроллере вывода.

Рассмотрим работу параллельного интерфейса ввода при реализации программы асинхронного обмена. Алгоритм асинхронного ввода так же прост, как и асинхронного вывода

1. Процессор проверяет наличие данных в регистре данных контроллера A1.

2. Если данные готовы (логическая 1 в регистре A2), то они передаются из регистра данных A1 на шину данных системного интерфейса и далее в регистр процессора или ячейку памяти микрокомпьютера. Иначе повторяется п. 1.

При параллельной передаче обеспечивается довольно высокая скорость обмена данными, которая ограничивается только быстродействием самого внешнего устройства.

4.2 ПОСЛЕДОВАТЕЛЬНАЯ ПЕРЕДАЧА ДАННЫХ

Использование последовательных линий связи для обмена данными с внешними устройствами возлагает на контроллеры внешнего устройства дополнительные по сравнению с контроллерами для параллельного обмена функции.

Во-первых, возникает необходимость преобразования формата данных из параллельного в последовательный при передаче информации на

внешнее устройство и преобразование из последовательного в параллельный формат при приеме данных с внешнего устройства.

Во-вторых, требуется реализовать соответствующий режиму работы внешнего устройства способ обмена данными: синхронный или асинхронный.

СИНХРОННЫЙ ПОСЛЕДОВАТЕЛЬНЫЙ ИНТЕРФЕЙС

Простой контроллер для синхронной передачи данных на внешнее устройство по последовательной линии связи (последовательный интерфейс) представлен на рис. 44.

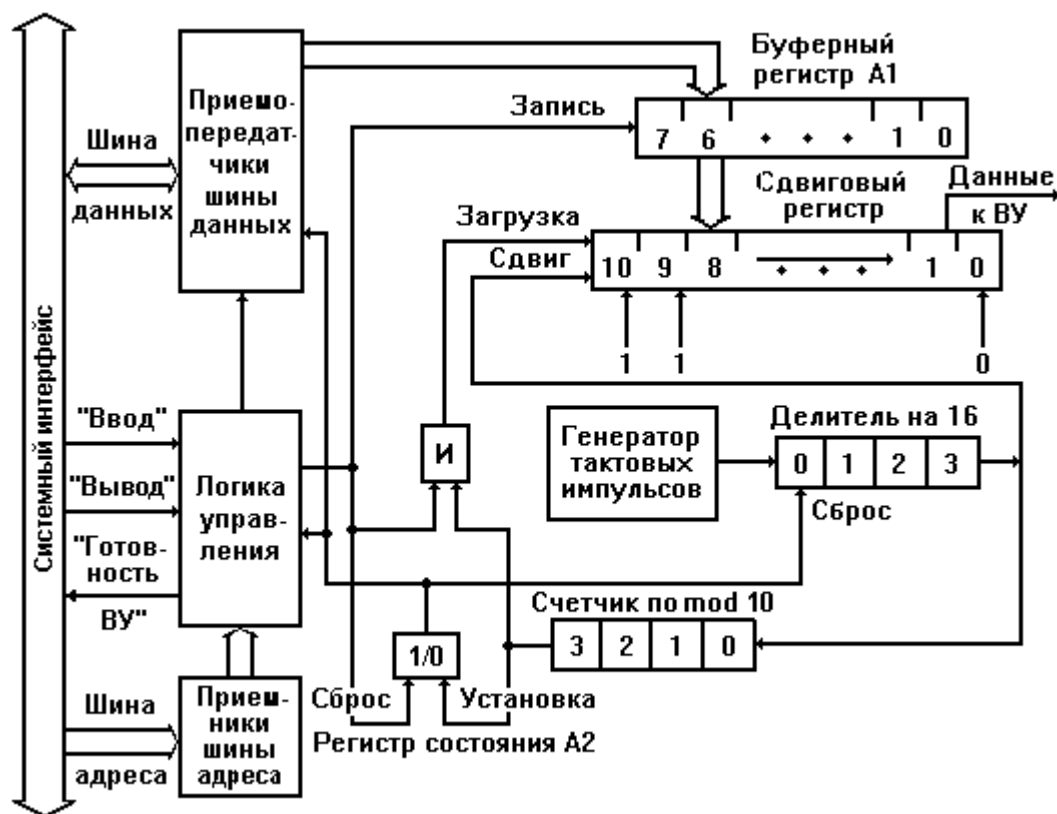


Рис. 44. Контроллер последовательной синхронной передачи

Восьмиразрядный адресуемый буферный регистр контроллера A1 служит для временного хранения байта данных до его загрузки в сдвиговый регистр. Запись байта данных в буферный регистр с шины данных системного интерфейса производится так же, как и в параллельном

интерфейсе, только при наличии единицы в одноразрядном адресуемом регистре состояния контроллера A2. Единица в регистре состояния указывает на готовность контроллера принять очередной байт в буферный регистр. Содержимое регистра A2 передается в процессор по одной из линий шины данных системного интерфейса и используется для формирования управляющего сигнала системного интерфейса "Готовность внешнего устройства". При записи очередного байта в буферный регистр A1 обнуляется регистр состояния A2.

Преобразование данных из параллельного формата, в котором они поступили в буферный регистр контроллера из системного интерфейса, в последовательный и передача их на линию связи производятся в сдвиговом регистре с помощью генератора тактовых импульсов и двоичного трехразрядного счетчика импульсов следующим образом.

Последовательная линия связи контроллера с внешним устройством подключается к выходу младшего разряда сдвигового регистра. По очередному тактовому импульсу содержимое сдвигового регистра сдвигается на один разряд вправо и в линию связи "Данные" выдается значение очередного разряда. Одновременно со сдвигом на внешнее устройство передается по отдельной линии "Синхронизация" тактовый импульс. Таким образом, каждый передаваемый по линии "Данные" бит информации сопровождается синхронизирующим сигналом по линии "Синхронизация", что обеспечивает его однозначное восприятие на приемном конце последовательной линии связи.

Количество переданных в линию тактовых сигналов, а следовательно, и переданных бит информации подсчитывается счетчиком тактовых импульсов. Как только содержимое счетчика становится равным 7, т. е. в линию переданы 8 бит (1 байт) информации, формируется управляющий сигнал "Загрузка", обеспечивающий запись в сдвиговый регистр очередного байта из буферного регистра. Этим же управляющим сигналом устанавливается в "1" регистр состояния. Очередным тактовым импульсом

счетчик будет сброшен в "0", и начнется очередной цикл выдачи восьми битов информации из сдвигового регистра в линию связи.

Синхронная последовательная передача отдельных битов данных на линию связи должна производиться без какого-либо перерыва, и следующий байт данных должен быть загружен в буферный регистр из системного интерфейса за время, не превышающее времени передачи восьми битов в последовательную линию связи.

При записи байта данных в буферный регистр обнуляется регистр состояния контроллера. Нуль в этом регистре указывает, что в линию связи передается байт данных из сдвигового регистра, а следующий передаваемый байт данных загружен в сдвиговый регистр.

Контроллер для последовательного синхронного приема данных из ВУ состоит из тех же компонентов, что и контроллер для синхронной последовательной передачи, за исключением генератора тактовых импульсов.

АСИНХРОННЫЙ ПОСЛЕДОВАТЕЛЬНЫЙ ИНТЕРФЕЙС

Организация асинхронного последовательного обмена данными с внешним устройством осложняется тем, что на передающей и приемной стороне последовательной линии связи используются настроенные на одну частоту, но физически разные генераторы тактовых импульсов и, следовательно, общая синхронизация отсутствует. Рассмотрим на примерах организацию контроллеров последовательных интерфейсов для последовательных асинхронных передачи и приема данных.

Простейший контроллер для асинхронной передачи данных на внешнее устройство по последовательной линии связи представлен на рис. 45. Он предназначен для передачи данных в формате с двумя стоповыми битами.

После передачи очередного байта данных в регистр состояния A2 записывается 1. Единичный выходной сигнал регистра A2 информирует

процессор о готовности контроллера к приему следующего байта данных и передаче его по линии связи на внешнее устройство. Этот же сигнал запрещает формирование импульсов со схемы выработки импульсов сдвига - делителя частоты сигналов тактового генератора на 16. Счетчик импульсов сдвига (счетчик по mod 10) находится в нулевом состоянии и его единичный выходной сигнал поступает на вентиль И, подготавливая цепь выработки сигнала загрузки сдвигового регистра.

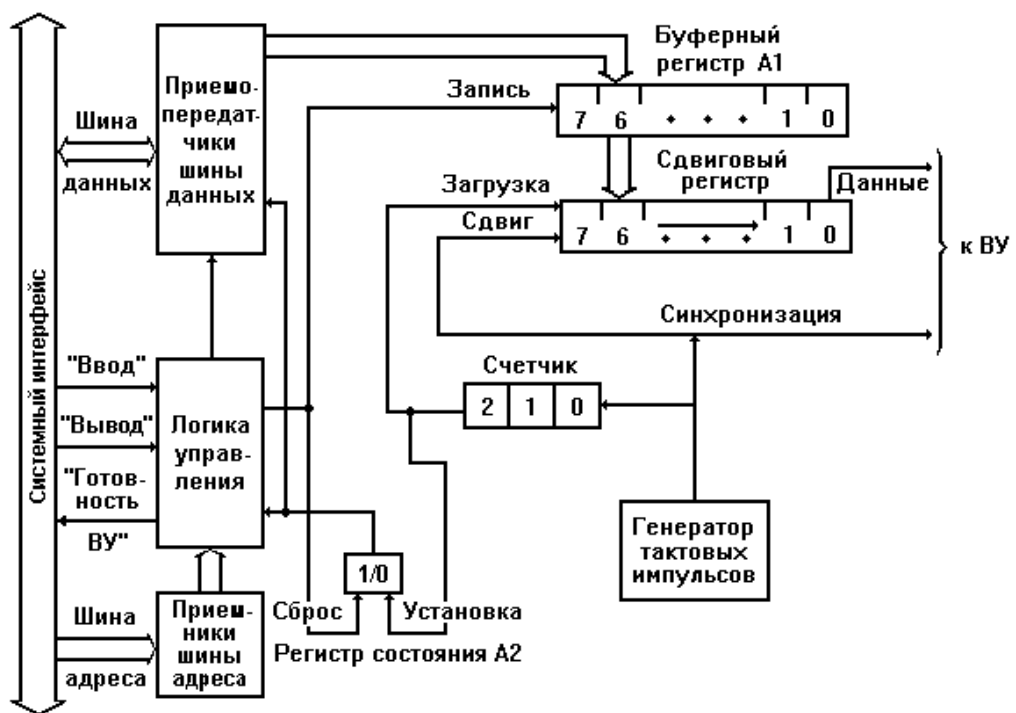


Рис. 45. Контроллер последовательной асинхронной передачи

Процесс передачи байта данных начинается с того, что процессор, выполняя команду "Вывод", выставляет этот байт на шине данных. Одновременно процессор формирует управляющий сигнал системного интерфейса "Вывод", по которому производятся запись передаваемого байта в буферный регистр А1, сброс регистра состояния А2 и формирование на вентиле И сигнала "Загрузка". Передаваемый байт переписывается в разряды 1, ..., 8 сдвигового регистра, в нулевой разряд сдвигового регистра записывается 0 (стартовый бит), а в разряды 9 и 10 - 1 (стоповые биты). Кроме того, снимается сигнал "Сброс" с делителя частоты, он начинает

накапливать импульсы генератора тактовой частоты и в момент приема шестнадцатого тактового импульса вырабатывает импульс сдвига.

На выходной линии контроллера "Данные" поддерживается состояние 0 (значение стартового бита) до тех пор, пока не будет выработан первый импульс сдвига. Импульс сдвига изменит состояние счетчика импульсов сдвига и перепишет в нулевой разряд сдвигового регистра первый информационный бит передаваемого байта данных. Состояние, соответствующее значению этого бита, будет поддерживаться на линии "Данные" до следующего импульса сдвига.

Аналогично будут переданы остальные информационные биты, первый стоповый бит и, наконец, второй стоповый бит, при передаче которого счетчик импульсов сдвига снова установится в нулевое состояние. Это приведет к записи 1 в регистр состояния A2. Единичный сигнал с выхода регистра A2 запретит формирование импульсов сдвига, а также информирует процессор о готовности к приему нового байта данных. После завершения передачи очередного кадра (стартового бита, информационного байта и двух стоповых бит) контроллер поддерживает в линии связи уровень логической единицы (значение второго стопового бита).

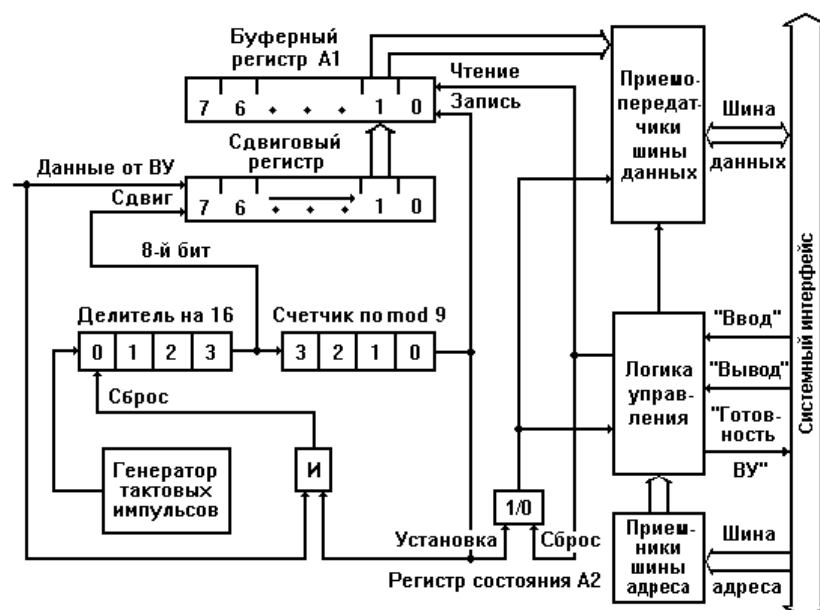


Рис. 46. Контроллер последовательного асинхронного приема

Уровень логической единицы поступает по линии "Данные" в контроллер для асинхронного приема данных (рис. 46). Этот уровень создает условия для выработки сигнала, запрещающего работу делителя частоты генератора тактовых импульсов. Действительно, после приема предыдущего байта данных счетчик импульсов сдвига (счетчик по mod 9) находится в нулевом состоянии и на вентиль И поступают два единичных сигнала: со счетчика сдвигов и из линии "Данные". На выходе вентилей И вырабатывается сигнал сброса делителя частоты сигналов тактового генератора, запрещающий формирование импульсов сдвига.

В момент смены стопового бита на стартовый бит (момент начала передачи нового кадра) на линии "Данные" появится уровень логического нуля и тем самым будет снят сигнал сброса с делителя частоты. Состояние 4-разрядного двоичного счетчика (делителя частоты) начнет изменяться. Когда на счетчике накопится значение 8, он выдаст сигнал, поступающий на входы сдвигового регистра и счетчика импульсов сдвига. Так как частота сигналов генератора тактовых импульсов приемника должна совпадать с частотой генератора тактовых импульсов передатчика, то сдвиг (считывание) бита произойдет примерно на середине временного интервала, отведенного на передачу бита данных, т. е. времени, необходимого для выработки шестнадцати тактовых импульсов. Это делается для уменьшения вероятности ошибки из-за возможного различия частот генераторов передатчика и приемника, искажения формы передаваемых сигналов (переходные процессы) и т. п. Следующий сдвиг произойдет после прохождения шестнадцати тактовых импульсов, т. е. на середине временного интервала передачи первого информационного бита.

При приеме в сдвиговый регистр девятого бита кадра (восьмого информационного бита) из него "выдвинется" стартовый бит и, следовательно, в сдвиговом регистре будет размещен весь принятый байт информации. В этот момент счетчик импульсов сдвига придет в нулевое состояние и на его выходе будет выработан единичный сигнал, по которому

содержимое сдвигового регистра переписывается в буферный регистр, в регистр состояния A2 запишется 1 и он будет информировать процессор об окончании приема очередного байта, вентиль И подготовится к выработке сигнала "Сброс" (этот сигнал сформируется после прихода первого стопового бита).

Получив сигнал готовности (1 в регистре A2), процессор выполнит команду "Ввод". При этом вырабатывается управляющий сигнал системного интерфейса "Ввод", по которому производится пересылка принятого байта данных из буферного регистра в процессор (сигнал "Чтение") и сброс регистра состояния A2.

1. СПОСОБЫ ОБМЕНА ИНФОРМАЦИЕЙ В МИКРОПРОЦЕССОРНОЙ СИСТЕМЕ

Применяются три режима ввода/вывода:

- программно-управляемый ввод-вывод информации (называемый также программным или нефорсированным вводом-выводом),
- ввод-вывод информации по прерываниям (форсированный ввод-вывод)
- прямой доступ к памяти.

Первый из них характеризуется тем, что инициирование и управление ввода-вывода информации осуществляется программой, выполняемой процессором, а внешние устройства играют сравнительно пассивную роль и сигнализируют только о своем состоянии, в частности, о готовности к операциям ввода/вывода. Во втором режиме ввод-вывод информации иницируется не процессором, а внешним устройством, генерирующим специальный сигнал прерывания. Реагируя на этот сигнал готовности устройства к передаче данных, процессор передает управление подпрограмме обслуживания устройства, вызвавшего прерывание. Действия, выполняемые этой подпрограммой, определяются пользователем, а непосредственными операциями вводом-выводом информации управляет

процессор. Наконец, в режиме прямого доступа к памяти, который используется, когда пропускной способности процессора недостаточно, действия процессора приостанавливаются, он отключается от системной шины и не участвует в передачах данных между основной памятью и быстродействующим внешним устройством. Заметим, что во всех вышеуказанных случаях основные действия, выполняемые системной шиной, подчиняются двум основным принципам.

1. В процессе взаимодействия любых двух электронных устройств одно из них обязательно выполняет активную, управляющую роль и является задатчиком, второе оказывается управляемым, исполнителем. Чаще всего задатчиком является процессор.

2. Другим важным принципом, заложенным в структуру интерфейса, является принцип квитирования (запроса - ответа): каждый управляющий сигнал, посланный задатчиком, подтверждается сигналом исполнителя. При отсутствии ответного сигнала исполнителя в течение заданного интервала времени формируется так называемый тайм-аут, задатчик фиксирует ошибку обмена и прекращает данную операцию.

Программно-управляемый ввод/вывод

Данный режим характеризуется тем, что все действия по вводу/выводу реализуются командами прикладной программы. Наиболее простыми эти действия оказываются для "всегда готовых" внешних устройств, например индикатора на светодиодах. При необходимости ввод-вывод информации в соответствующем месте программы используются команды IN или OUT. Такая передача данных называется синхронным или безусловным вводом-выводом информации.

Однако для большинства внешних устройств до выполнения операций ввод-вывод информации надо убедиться в их готовности к обмену, т.е. ввод-вывод информации является асинхронным. Общее состояние устройства характеризуется флагом готовности READY, называемым также флагом готовности/занятости (READY/BUSY). Иногда состояния готовности и

занятости идентифицируются отдельными флагами READY и BUSY, входящими в слово состояния устройства.

Процессор проверяет флаг готовности с помощью одной или нескольких команд. Если флаг установлен, то инициируются собственно ввод или вывод одного или нескольких слов данных. Когда же флаг сброшен, процессор выполняет цикл из 2-3 команд с повторной проверкой флага READY до тех пор, пока устройство не будет готово к операциям ввода-вывода (рис. 47). Данный цикл называется циклом ожидания готовности ВУ и реализуется в различных процессорах по-разному.



Рис. 47. Цикл программного ожидания готовности внешнего устройства

Основной недостаток программного ввода-вывода связан с непроизводительными потерями времени процессора в циклах ожидания. К достоинствам следует отнести простоту его реализации, не требующей дополнительных аппаратных средств.

Организация прерываний

Одной из разновидностей программно-управляемого обмена данными с внешнего устройства в микропроцессорную систему является обмен с прерыванием программы, отличающийся от асинхронного программно-управляемого обмена тем, что переход к выполнению команд, физически реализующих обмен данными, осуществляется с помощью специальных аппаратных средств. Команды обмена данными в этом случае выделяют в

отдельный программный модуль - подпрограмму обработки прерывания. Задачей аппаратных средств обработки прерывания в процессоре как раз и является приостановка выполнения одной программы (ее еще называют основной программой) и передача управления подпрограмме обработки прерывания. Действия, выполняемые при этом процессором, как правило, те же, что и при обращении к подпрограмме. Только при обращении к подпрограмме они инициируются командой, а при обработке прерывания - управляющим сигналом от внешнего устройства, который называют "Запрос на прерывание" или "Требование прерывания".

Эта важная особенность обмена с прерыванием программы позволяет организовать обмен данными с внешнего устройства в произвольные моменты времени, не зависящие от программы, выполняемой в микропроцессорной системе. Таким образом, появляется возможность обмена данными с внешнего устройства в реальном масштабе времени, определяемом внешней по отношению к микропроцессорной системе средой. Обмен с прерыванием программы существенным образом экономит время процессора, затрачиваемое на обмен. Это происходит за счет того, что исчезает необходимость в организации программных циклов ожидания готовности внешнего устройства, на выполнение которых тратится значительное время, особенно при обмене с медленными внешними устройствами.

Прерывание программы по требованию внешнего устройства не должно оказывать на прерванную программу никакого влияния кроме увеличения времени ее выполнения за счет приостановки на время выполнения подпрограммы обработки прерывания. Поскольку для выполнения подпрограммы обработки прерывания используются различные регистры процессора (счетчик команд, регистр состояния и т.д.), то информацию, содержащуюся в них в момент прерывания, необходимо сохранить для последующего возврата в прерванную программу.

Обычно задача сохранения содержимого счетчика команд и регистра состояния процессора возлагается на аппаратные средства обработки прерывания. Сохранение содержимого других регистров процессора, используемых в подпрограмме обработки прерывания, производится непосредственно в подпрограмме. Отсюда следует достаточно очевидный факт: чем больший объем информации о прерванной программе сохраняется программным путем, тем больше время реакции микропроцессорной системы на сигнал прерывания, и наоборот. Предпочтительными с точки зрения повышения производительности являются уменьшение числа команд, обеспечивающих сохранение информации о прерванной программе, и реализация этих функций аппаратными средствами.

Формирование сигналов прерываний - запросов внешнего устройства на обслуживание происходит в контроллерах соответствующих внешних устройств. В простейших случаях в качестве сигнала прерывания может использоваться сигнал "Готовность внешнего устройства", поступающий из контроллера внешнего устройства в системный интерфейс. Однако такое решение обладает существенным недостатком - процессор не имеет возможности управлять прерываниями, т. е. разрешать или запрещать их для отдельных внешних устройств. В результате организация обмена данными в режиме прерывания с несколькими внешними устройствами существенно усложняется.

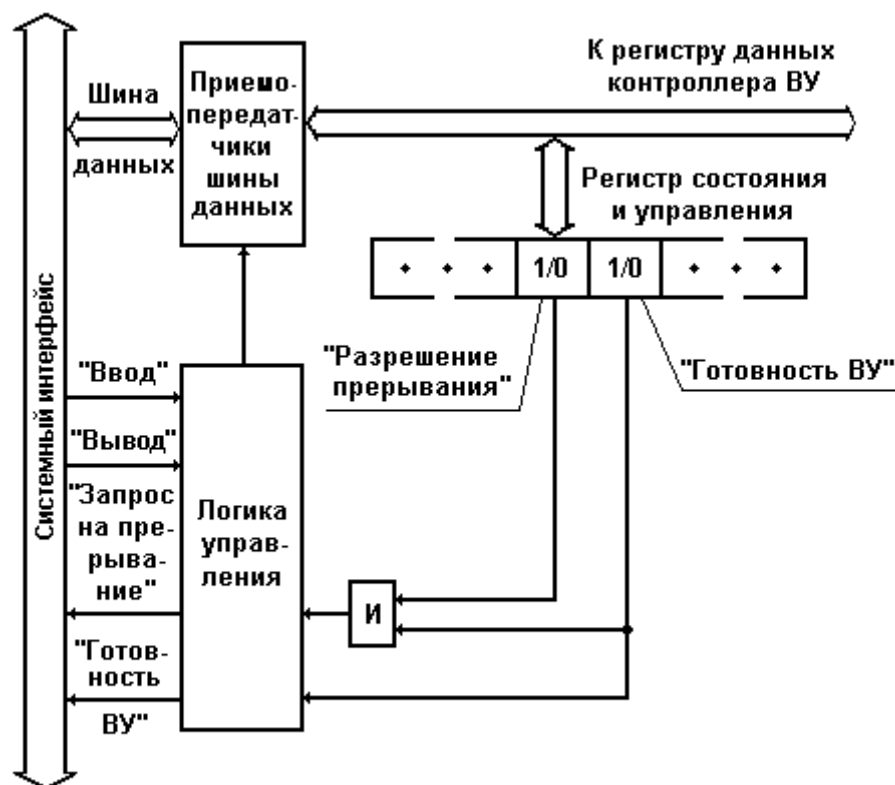


Рис. 48. Фрагмент блок-схемы контроллера внешнего устройства с разрядом "Разрешение прерывания" в регистре состояния и управления

Для решения этой проблемы регистр состояния и управления контроллера внешнего устройства (рис. 48) дополняют еще одним разрядом - "Разрешение прерывания". Запись 1 или 0 в разряд "Разрешение прерывания" производится программным путем по одной из линий шины данных системного интерфейса. Управляющий сигнал системного интерфейса "Запрос на прерывание" формируется с помощью схемы совпадения только при наличии единиц в разрядах "Готовность ВУ" и "Разрешение прерывания" регистра состояния и управления контроллера.

Обычно используется одноуровневая система прерываний, т. е. сигналы "Запрос на прерывание" от всех внешних устройств поступают на один вход процессора. Поэтому возникает проблема идентификации внешнего устройства, запросившего обслуживание, и реализации заданной очередности (приоритета) обслуживания внешних устройств при одновременном поступлении нескольких сигналов прерывания. Существуют

два основных способа идентификации внешних устройств, запросивших обслуживания:

- программный опрос регистров состояния (разряд "Готовность внешнего устройства") контроллеров всех внешних устройств;
- использование векторов прерывания.

Организация прерываний с программным опросом готовности предполагает наличие в памяти микропроцессорной системы единой подпрограммы обслуживания прерываний от всех внешних устройств. Структура такой подпрограммы приведена на рис. 49.

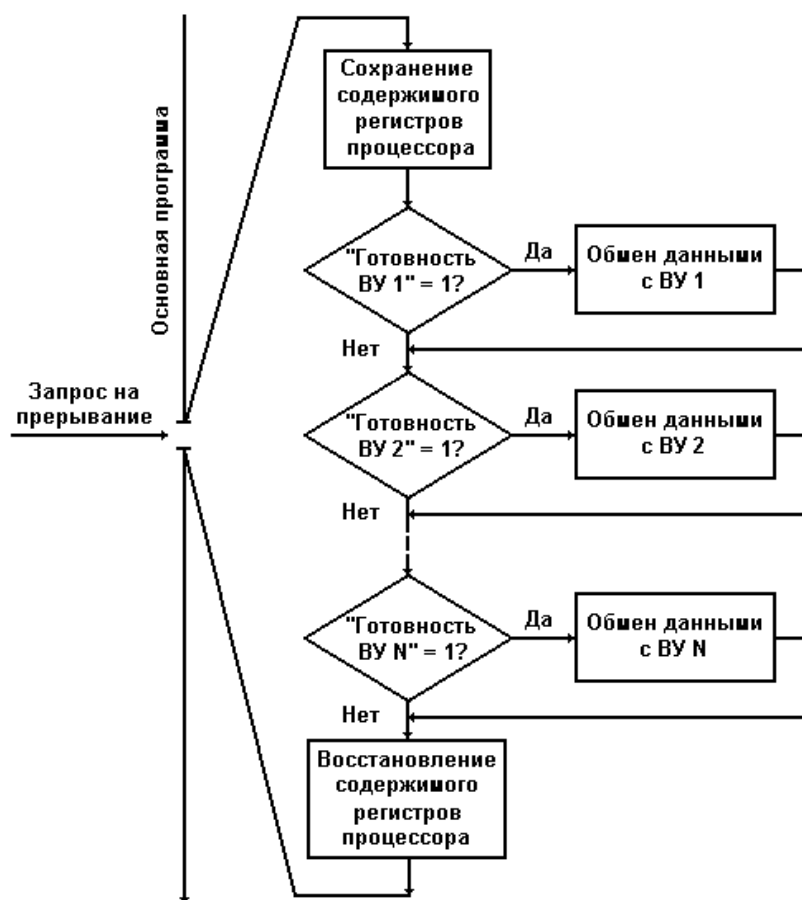


Рис. 49. Структура единой программы обработки прерываний и ее связь с основной программой

Обслуживание внешнего устройства с помощью единой подпрограммы обработки прерываний производится следующим образом. В конце последнего машинного цикла выполнения очередной команды основной программы процессор проверяет наличие требования прерывания от

внешнего устройства. Если сигнал прерывания есть и в процессоре прерывание разрешено, то процессор переключается на выполнение подпрограммы обработки прерываний.

После сохранения содержимого регистров процессора, используемых в подпрограмме, начинается последовательный опрос регистров состояния контроллеров всех внешних устройств, работающих в режиме прерывания. Как только подпрограмма обнаружит готовое к обмену внешнее устройство, сразу выполняются действия по его обслуживанию. Завершается подпрограмма обработки прерывания после опроса готовности всех внешних устройств и восстановления содержимого регистров процессора.

Приоритет внешнего устройства в микропроцессорной системе с программным опросом готовности внешнего устройства однозначно определяется порядком их опроса в подпрограмме обработки прерываний. Чем раньше в подпрограмме опрашивается готовность внешнего устройства, тем меньше время реакции на его запрос и выше приоритет. Необходимость проверки готовности всех внешних устройств существенно увеличивает время обслуживания тех внешних устройств, которые опрашиваются последними. Это является основным недостатком рассматриваемого способа организации прерываний. Поэтому обслуживание прерываний с опросом готовности внешнего устройства используется только в тех случаях, когда отсутствуют жесткие требования на время обработки сигналов прерывания внешних устройств.

Организация системы прерываний с использованием векторов прерываний позволяет устранить указанный недостаток. При такой организации системы прерываний внешних устройств, запросившее обслуживания, само идентифицирует себя с помощью вектора прерывания - адреса ячейки основной памяти микропроцессорной системы, в которой хранится либо первая команда подпрограммы обслуживания прерывания данного внешнего устройства, либо адрес начала такой подпрограммы. Таким образом, процессор, получив вектор прерывания, сразу переключается

на выполнение требуемой подпрограммы обработки прерывания. В микропроцессорной системе с векторной системой прерывания каждое внешнее устройство должно иметь собственную подпрограмму обработки прерывания.

Различают векторные системы с интерфейсным и внеинтерфейсным вектором. В первом случае вектор прерывания (или его адрес) формирует контроллер внешнего устройства, запросившего обслуживания, во втором - контроллер прерываний, общий для всех устройств, работающих в режиме прерываний (IBM-совместимые персональные компьютеры).

Рассмотрим организацию векторной системы с интерфейсным вектором. Вектор прерывания (или его адрес) выдается контроллером не одновременно с запросом на прерывание, а только по разрешению процессора, как это реализовано в схеме на рис. 50. Это делается для того, чтобы исключить одновременную выдачу векторов прерывания от нескольких внешних устройств. В ответ на сигнал контроллера внешнего устройства "Запрос на прерывание" процессор формирует управляющий сигнал "Предоставление прерывания (вх.)", который разрешает контроллеру внешнего устройства, запросившему обслуживание, выдачу вектора прерывания в шину адреса системного интерфейса. Для этого в контроллере используются регистр вектора прерывания и схема совпадения ИЗ. Регистр вектора прерывания обычно реализуется с помощью перемычек или переключателей, что позволяет пользователю устанавливать для конкретных внешних устройств требуемые значения векторов прерывания.

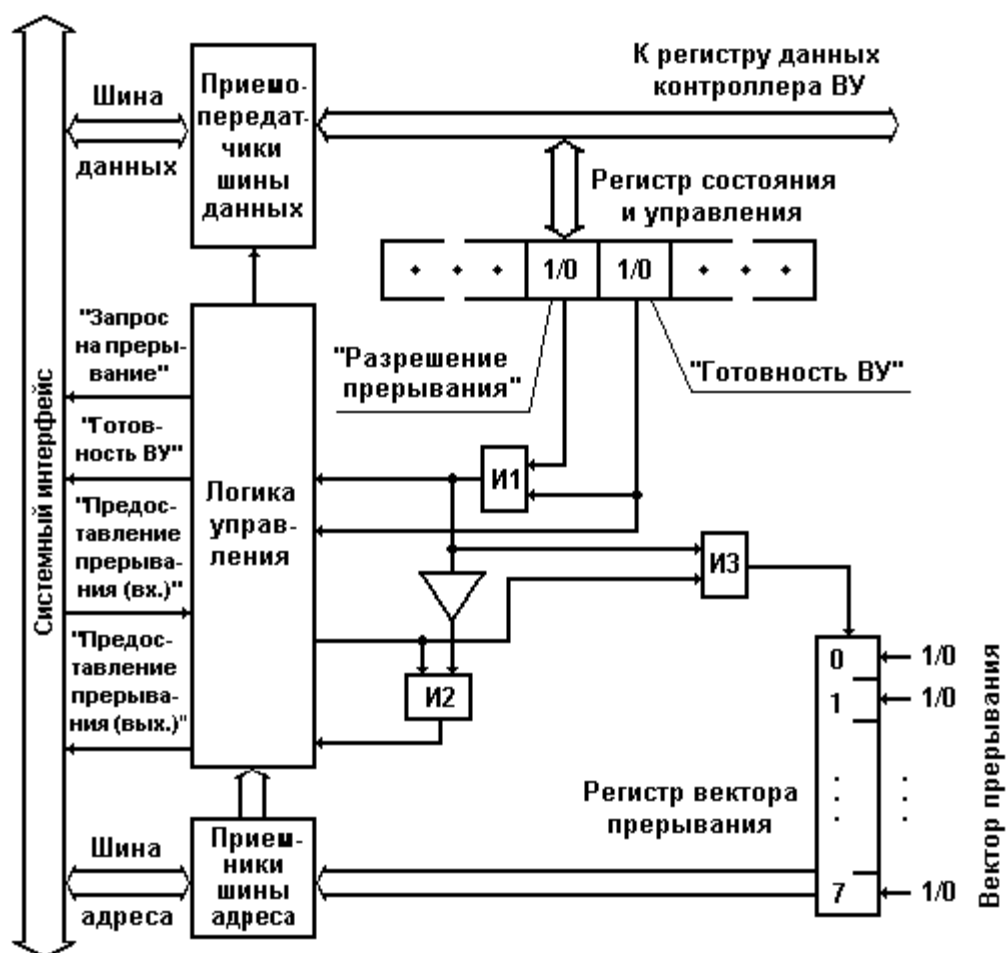


Рис. 50. Формирование векторов прерывания в контроллере ВУ

Управляющий сигнал "Предоставление прерывания (вых.)" формируется в контроллере ВУ с помощью схемы совпадения И2. Этот сигнал используется для организации последовательного аппаратного опроса готовности внешних устройств и реализации тем самым требуемых приоритетов внешних устройств. Процессор при поступлении в него по общей линии системного интерфейса "Запрос на прерывание" сигнала прерывания формирует управляющий сигнал "Предоставление прерывания (вх.)", который поступает сначала в контроллер внешнего устройства с наивысшим приоритетом (рис. 51). Если это устройство не требовало обслуживания, то его контроллер пропускает сигнал "Предоставление прерывания" на следующий контроллер, иначе дальнейшее распространение сигнала прекращается и контроллер выдает вектор прерывания на адресно-информационную шину.



Рис. 51. Реализация приоритетов ВУ в микроЭВМ с векторной системой прерываний с интерфейсным вектором (ППР (вх.) - "Предоставление прерывания (входной)"; "ППР (вых.) - Предоставление прерывания (выходной)")

Аппаратный опрос готовности внешних устройств производится гораздо быстрее, нежели программный. Но если обслуживания запросили одновременно два или более внешних устройства, обслуживание менее приоритетных внешних устройств будет отложено на время обслуживания более приоритетных, как и в системе прерывания с программным опросом.

Векторная система с внеинтерфейсным вектором прерывания используется в IBM-совместимых персональных компьютерах. В этих компьютерах контроллеры внешних устройств не имеют регистров для хранения векторов прерывания, а для идентификации устройств, запросивших обслуживания, используется общий для всех внешних устройств контроллер прерываний. Ниже приведен пример контроллера прерываний INTEL 8259A.

БИС программируемого контроллера прерываний (ПКП) представляет собой устройство, реализующее до восьми уровней запросов на прерывания с возможностью программного маскирования и изменения порядка обслуживания прерываний. За счет каскадного включения БИС ПКП число уровней прерывания может быть расширено до 64 (в архитектуре персонального компьютера IBM PC AT - 16).

Структурная схема ПКП приведена на рисунке 52.

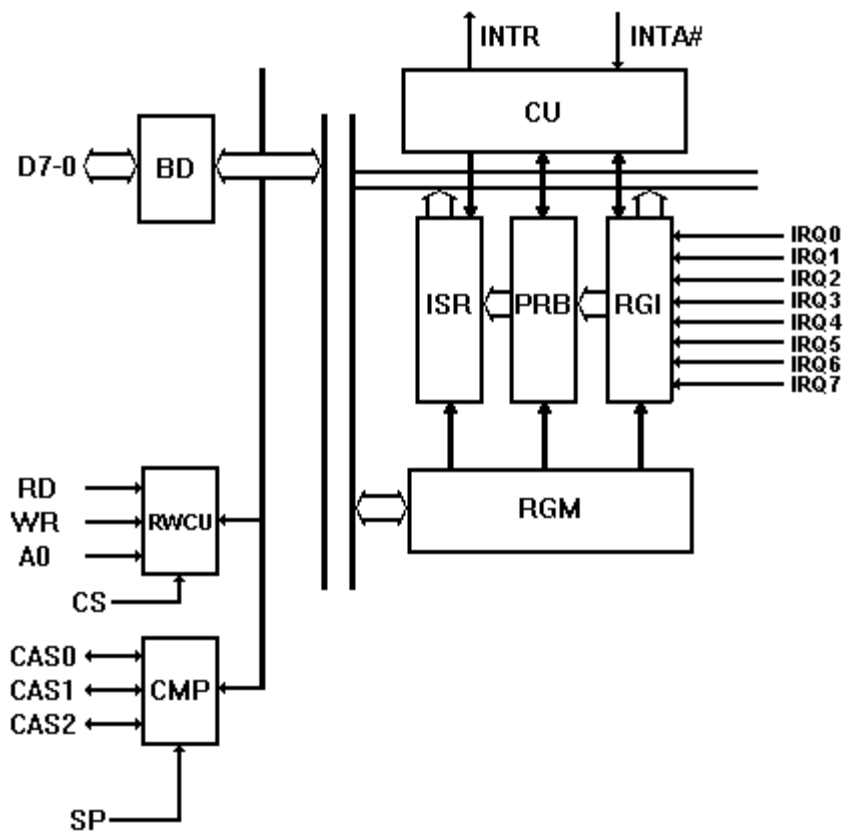


Рис. 52. Контроллер прерываний Intel 8259A

В состав БИС входят:

RGI - регистр запретов прерываний; хранит все уровни, на которые поступают запросы IRQx;

PRB - схема принятия решений по приоритетам; схема идентифицирует приоритет запросов и выбирает запрос с наивысшим приоритетом;

ISR - регистр обслуживаемых прерываний; сохраняет уровни запросов прерываний, находящиеся на обслуживании ПКП;

RGM - регистр маскирования прерываний; обеспечивает запрещение одной или нескольких линий запросов прерывания;

BD - буфер данных; предназначен для сопряжения ПКП с системной шиной данных;

RWCU - блок управления записью/чтением; принимает управляющие сигналы от микропроцессора и задает режим функционирования ПКП;

CMP - схема каскадного буфера-компаратора; используется для включения

в систему нескольких ПКП;

CU - схема управления; вырабатывает сигналы прерывания и формирует трехбайтовую команду CALL для выдачи на шину данных.

Установка ПКП в исходное состояние и "настройка" его на определенный режим обслуживания прерываний происходит с помощью двух типов команд: команд инициализации (ICW) и команд управления операциями (OCW).

Программируемый контроллер прерываний (ПКП) имеет 16 входов запросов прерываний (IRQ 0 - IRQ 15). Контроллер состоит из двух каскадно включенных контроллеров - выход INTR (запрос на прерывание) второго контроллера подключен ко входу IRQ 2 первого контроллера. В качестве примера отметим, что к линии IRQ 0 подключен системный таймер, к линии IRQ 1 - клавиатура, к линии IRQ 8 - часы реального времени и т.д.

Упрощенная схема взаимодействия контроллера прерываний с процессором и контроллером шины имеет следующий вид.

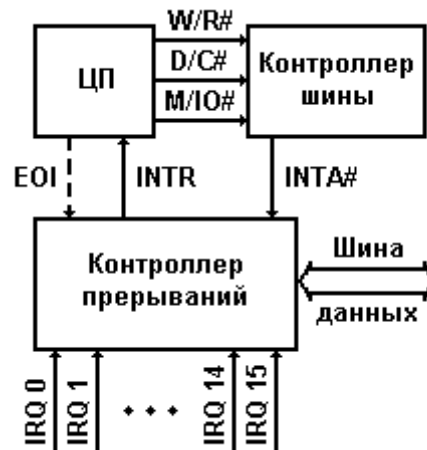


Рис. 53. Упрощенная схема взаимодействия контроллера прерываний с процессором и контроллером шины в IBM-совместимых персональных компьютерах класса AT

Эта схема функционирует следующим образом. Пусть в некоторый момент времени контроллер клавиатуры с помощью единичного сигнала по линии IRQ 1 известил контроллер прерываний о своей готовности к обмену. В ответ на запрос контроллер прерываний генерирует сигнал INTR (запрос на прерывание) и посылает его на соответствующий вход процессора.

Процессор, если маскируемые прерывания разрешены (т.е. установлен флаг прерываний IF в регистре флагов процессора), посылает на контроллер шины сигналы R# - чтение, C# - управление и IO# - ввод/вывод, определяющие тип цикла шины. Контроллер шины, в свою очередь, генерирует два сигнала подтверждения прерывания INTA# и направляет их на контроллер прерываний. По второму импульсу контроллер прерываний выставляет на шину данных восьмибитный номер вектора прерывания, соответствующий данной линии IRQ.

В режиме реального адреса ("реальном" режиме) векторы прерываний хранятся в таблице векторов прерываний, которая находится в первом килобайте оперативной памяти. Под каждый вектор отведено 4 байта (2 байта под адрес сегмента и 2 байта под смещение), т.е. в таблице может содержаться 256 векторов. Адрес вектора в таблице - номер вектора * 4.

Далее процессор считывает номер вектора прерывания. Сохраняет в стеке содержимое регистра флагов, сбрасывает флаг прерываний IF и помещает в стек адрес возврата в прерванную программу (регистры CS и IP). После этого процессор извлекает из таблицы векторов прерываний адрес подпрограммы обработки прерываний для данного устройства и приступает к ее выполнению.

Процедура обработки аппаратного прерывания должна завершаться командой конца прерывания EOI (End of Interruption), посылаемой контроллеру прерываний. Для этого необходимо записать байт 20h в порт 20h (для первого контроллера) и в порт A0h (для второго).

В IBM PC/XT/AT используется режим прерываний с фиксированными приоритетами. Высшим приоритетом обладает запрос по линии IRQ 0, низшим - IRQ 7. Так как второй контроллер подключен к линии IRQ 2 первого контроллера, то приоритеты линий IRQ в порядке убывания приоритета располагаются следующим образом: IRQ 0, IRQ 1, IRQ 8 - IRQ 15, IRQ 3 - IRQ 7. Если запрос на обслуживание посылают одновременно два устройства с разными приоритетами, то контроллер обслуживает запрос с

большим приоритетом, а запрос с меньшим приоритетом блокирует.
Блокировка сохраняется до получения команды EOI.

Организация прямого доступа к памяти

Одним из способов обмена данными с ВУ является обмен в режиме прямого доступа к памяти (ПДП). В этом режиме обмен данными между ВУ и памятью происходит без участия процессора. Обменом в режиме ПДП управляет не программа, выполняемая процессором, а электронные схемы, внешние по отношению к процессору. Обычно схемы, управляющие обменом в режиме ПДП, размещаются или в специальном контроллере, который называется контроллером прямого доступа к памяти, или в контроллере самого ВУ.

Обмен данными в режиме ПДП позволяет использовать быстродействующие внешние запоминающие устройства. ПДП может обеспечить время обмена одним байтом данных между памятью и ВЗУ, равное циклу обращения к памяти.

Для реализации режима прямого доступа к памяти необходимо обеспечить непосредственную связь контроллера ПДП и памяти. Для этой цели можно было бы использовать специально выделенные шины адреса и данных, связывающие контроллер ПДП с основной памятью. Но такое решение нельзя признать оптимальным, так как это приведет к значительному усложнению микропроцессорной системы в целом, особенно при подключении нескольких внешних запоминающих устройств. В целях сокращения количества линий в шинах контроллер ПДП подключается к памяти посредством шин адреса и данных системного интерфейса. При этом возникает проблема совместного использования шин системного интерфейса процессором и контроллером ПДП. Можно выделить два основных способа ее решения: реализация обмена в режиме ПДП с "захватом цикла" и в режиме ПДП с блокировкой процессора.

Существуют две разновидности прямого доступа к памяти с "захватом цикла". Наиболее простой способ организации ПДП состоит в том, что для обмена используются те машинные циклы процессора, в которых он не обменивается данными с памятью. В такие циклы контроллер ПДП может

обмениваться данными с памятью, не мешая работе процессора. Однако возникает необходимость выделения таких циклов, чтобы не произошло временного перекрытия обмена ПДП с операциями обмена, инициируемыми процессором. В некоторых процессорах формируется специальный управляющий сигнал, указывающий циклы, в которых процессор не обращается к системному интерфейсу. При использовании других процессоров для выделения таких циклов необходимо применение в контроллерах ПДП специальных селектирующих схем, что усложняет их конструкцию. Применение рассмотренного способа организации ПДП не снижает производительности, но при этом обмен в режиме ПДП возможен только в случайные моменты времени одиночными байтами или словами.

Более распространенным является ПДП с "захватом цикла" и принудительным отключением процессора от шин системного интерфейса. Для реализации такого режима ПДП системный интерфейс дополняется двумя линиями для передачи управляющих сигналов "Требование прямого доступа к памяти" (ТПДП) и "Предоставление прямого доступа к памяти" (ППДП).

Управляющий сигнал ТПДП формируется контроллером прямого доступа к памяти. Процессор, получив этот сигнал, приостанавливает выполнение очередной команды, не дожидаясь ее завершения, выдает на системный интерфейс управляющий сигнал ППДП и отключается от шин системного интерфейса. С этого момента все шины системного интерфейса управляются контроллером ПДП. Контроллер ПДП, используя шины системного интерфейса, осуществляет обмен одним байтом или словом данных с памятью МПС и затем, сняв сигнал ТПДП, возвращает управление системным интерфейсом процессору. Как только контроллер ПДП будет готов к обмену следующим байтом, он вновь "захватывает" цикл процессора и т.д. В промежутках между сигналами ТПДП процессор продолжает выполнять команды программы. Тем самым выполнение программы замедляется, но в меньшей степени, чем при обмене в режиме прерываний.

Обмен данными с внешними устройствами в режиме ПДП всегда требует предварительной подготовки, а именно: для каждого внешнего устройства необходимо выделить область памяти, используемую при обмене, и указать ее размер, т.е. количество записываемых в память или читаемых из памяти байт (слов) информации. Следовательно, контроллер ПДП должен обязательно иметь в своем составе регистр адреса и счетчик байт (слов). Перед началом обмена информацией с внешним устройством в режиме ПДП процессор должен выполнить программу загрузки. Эта программа обеспечивает запись в указанные регистры контроллера ПДП начального адреса выделенной внешнему устройству памяти и ее размера в байтах или словах в зависимости от того, какими порциями информации ведется обмен. Сказанное не относится к начальной загрузке программ в память в режиме ПДП. В этом случае содержимое регистра адреса и счетчика байт слов устанавливается переключателями или перемычками непосредственно на плате контроллера.

Блок-схема простого контроллера ПДП, обеспечивающего ввод данных в память по инициативе ВУ в режиме ПДП "Захват цикла", приведена на рис. 54.

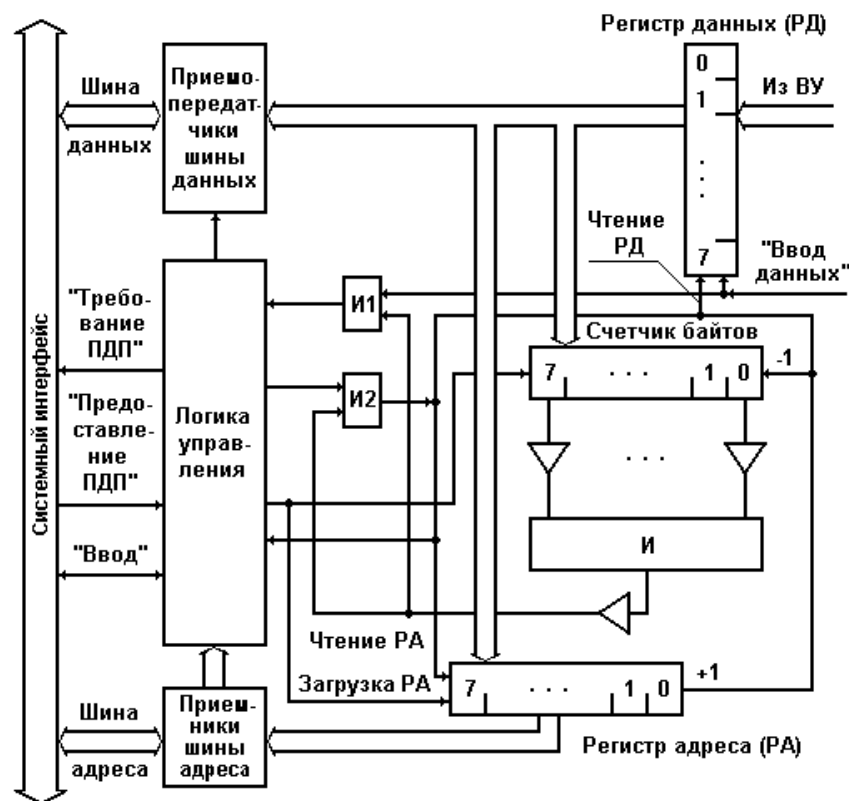


Рис. 54. Контроллер ПДП для ввода данных с внешнего устройства в режиме "Захват цикла" и отключением процессора от шин системного интерфейса

Перед началом очередного сеанса ввода данных с внешнего устройства процессор загружает в регистры его контроллера следующую информацию: в счетчик байт - количество принимаемых байт данных, а в регистр адреса - начальный адрес области памяти для вводимых данных. Тем самым контроллер подготавливается к выполнению операции ввода данных с внешнего устройства в память микропроцессорной системы в режиме ПДП.

Байты данных с внешнего устройства поступают в регистр данных контроллера в постоянном темпе. При этом каждый байт сопровождается управляющим сигналом с внешнего устройства "Ввод данных", который обеспечивает запись байта данных в регистр данных контроллера. По этому же сигналу и при ненулевом состоянии счетчика байт контроллер формирует сигнал ТПДП. По ответному сигналу процессора ППДП контроллер выставляет на шины адреса и данных системного интерфейса содержимое своих регистров адреса и данных соответственно. Формируя управляющий

сигнал "Вывод", контроллер ПДП обеспечивает запись байта данных из своего регистра данных в память микропроцессорной системы. Сигнал ППДП используется в контроллере и для модификации счетчика байт и регистра адреса. По каждому сигналу ППДП из содержимого счетчика байт вычитается единица, и как только содержимое счетчика станет равно нулю, контроллер прекратит формирование сигналов "Требование прямого доступа к памяти".

На примере простого контроллера ПДП мы рассмотрели только процесс подготовки контроллера и непосредственно передачу данных в режиме ПДП. На практике любой сеанс обмена данными с ВУ в режиме ПДП всегда инициируется программой, выполняемой процессором, и включает два следующих этапа.

1. На этапе подготовки внешнего устройства к очередному сеансу обмена информацией процессор в режиме программно-управляемого обмена опрашивает состояние внешнего устройства (проверяет его готовность к обмену) и посылает на внешнее устройство команды, обеспечивающие подготовку его к обмену. Такая подготовка может сводиться, например, к перемещению головок на требуемую дорожку в накопителе на жестком диске. Затем выполняется загрузка регистров контроллера ПДП. На этом подготовка к обмену в режиме ПДП завершается и процессор переключается на выполнение другой программы.

2. Обмен данными в режиме ПДП начинается после завершения подготовительных операций на внешнем устройстве по инициативе либо самого устройства, как это было рассмотрено выше, либо процессора. В этом случае контроллер ПДП необходимо дополнить регистром состояния и управления, содержимое которого будет определять режим работы контроллера ПДП. Один из разрядов этого регистра будет инициировать обмен данными с внешним устройством. Загрузка информации в регистр состояния и управления контроллера ПДП производится программным путем.

Наиболее распространенным является обмен в режиме прямого доступ к памяти с блокировкой процессора. Он отличается от ПДП с "захватом цикла" тем, что управление системным интерфейсом передается контроллеру ПДП не на время обмена одним байтом, а на время обмена блоком данных. Такой режим ПДП используется в тех случаях, когда время обмена одним байтом с ВУ сопоставимо с циклом системной шины.

В микропроцессорной системе можно использовать несколько внешних устройств, работающих в режиме ПДП. Предоставление таким устройствам шин системного интерфейса для обмена данными производится на приоритетной основе. Приоритеты внешних устройств реализуются так же, как и при обмене данными в режиме прерывания, но вместо управляющих сигналов "Требование прерывания" и "Предоставление прерывания" используются сигналы "Требование прямого доступа" и "Предоставление прямого доступа", соответственно.